ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY

College of Engineering

Department of Electrical and Computer Engineering

Real-Time Public Transport Tracking and Notification System

A Thesis Report Submitted to the Department of Electrical and Computer Engineering in Partial Fulfillment of the Requirements for the Award of Bachelor of Science Degree in Electrical and Computer Engineering (Computer Engineering Focus Area)

By:

Tinbite Yonas ETS 1559/14
Yafet Hailesilasse ETS 1616/14
Yohannes Desalegn ETS 1705/14
Yohannis Abebe ETS 1721/14

Advisor: Mr. Asamnew

May, 2026
Addis Ababa, Ethiopia

Declaration

We, the undersigned, declare that this thesis report entitled "Real-Time Public Transport Tracking and Notification System" is the result of our own original research work under the supervision of Mr. Asamnew. It has not been submitted for any degree or qualification in any university or institution before. All sources of information used have been duly acknowledged.

Tinbite Yonas Signature: _____ Date: _____

Yafet Hailesilasse Signature: _____ Date: _____

Yohannes Desalegn Signature: _____ Date: _____

Yohannis Abebe Signature: _____ Date: _____

Advisor Approval:
Name: Mr. Asamnew Signature: _____ Date: _____

Acknowledgments

First and foremost, we express our deepest gratitude to the Almighty God for granting us the strength, patience, and wisdom to complete this thesis work. Without His guidance, none of this would have been possible.

We extend our sincere and heartfelt appreciation to our thesis advisor, Mr. Asamnew, for his invaluable guidance, constructive feedback, and unwavering support throughout the course of this research. His expertise in embedded systems and IoT technologies was instrumental in shaping the direction of this work. We are particularly grateful for his patience in reviewing our work and providing detailed comments that significantly improved the quality of this thesis.

We would like to thank the Department of Electrical and Computer Engineering at Addis Ababa Science and Technology University for providing us with the academic foundation and laboratory resources necessary to carry out this research. Special thanks go to the department head and all faculty members who contributed to our education over the past four years.

Our gratitude also extends to the Anbessa City Bus Service Enterprise for granting us permission to install our tracking devices on their vehicles and conduct field testing on Route 12. The cooperation of the bus drivers and depot managers was essential to the success of our deployment.

We are deeply thankful to our families for their unconditional love, encouragement, and financial support throughout our academic journey. Their sacrifices have made it possible for us to pursue higher education and complete this research work.

Finally, we thank our fellow students and friends who participated in beta testing, provided feedback on the mobile application, and offered moral support during the challenging phases of this project.

Abstract

Public transportation in Addis Ababa has long been characterized by unpredictable schedules, overcrowded vehicles, and a near-total absence of real-time information for commuters. Passengers routinely wait at bus stops for indefinite periods with no reliable way to know when the next bus will arrive or how full it might be. This thesis presents the design, implementation, and evaluation of a comprehensive Real-Time Public Transport Tracking and Notification System that directly addresses these challenges through the integration of Internet of Things (IoT) hardware, modern web technologies, and machine learning techniques.

The system architecture comprises three interconnected tiers. At the edge, an ESP32-CAM microcontroller paired with a NEO-6M GPS module serves as the onboard tracking device, capturing real-time location coordinates and interior bus images at regular intervals. These data are transmitted over Wi-Fi to a FastAPI-based backend server, which processes telemetry feeds, runs computer vision algorithms for passenger density estimation, and stores everything in a PostgreSQL database with PostGIS spatial extensions. Redis caching ensures that frequently accessed route and position data are served with minimal latency. On the application side, a Flutter-based mobile app gives passengers live bus locations, estimated arrival times, and service notifications, while a Next.js web dashboard provides drivers and fleet administrators with route monitoring and operational analytics.

A key contribution of this work lies in the integration of machine learning for delay prediction. A RandomForest regression model, trained on historical trip data with features including time of day, day of week, peak hour indicators, stop identifiers, and occupancy levels, achieves a mean absolute error of 1.8 minutes on holdout test data. The computer vision pipeline uses OpenCV's HOG (Histogram of Oriented Gradients) people detector to estimate passenger counts from ESP32-CAM images, mapping these to three density levels (low, medium, high) that inform both the ETA engine and the passenger-facing occupancy display.

Field testing over three weeks with five buses operating on Route 12 demonstrated a GPS positioning accuracy of 2.5 meters (circular error probable), a telemetry transmission success rate of 98.7%, and a notification delivery rate of 96.2%. The system achieved 99.2% uptime during the test period, and user surveys indicated a 12% reduction in perceived wait times among passengers who used the mobile application. The total hardware cost per vehicle was approximately $43, representing an 80% cost reduction compared to commercial fleet tracking solutions.

Keywords: GPS tracking, Internet of Things, public transportation, real-time systems, ESP32-CAM, NEO-6M, Flutter, FastAPI, machine learning, delay prediction, passenger density estimation, computer vision.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background of the Study

Urban transportation sits at the heart of every functioning city. When it works well, it connects people to jobs, education, healthcare, and social opportunities. When it fails, it becomes a daily source of frustration, economic loss, and environmental harm. In Addis Ababa, Ethiopia's capital and one of Africa's fastest-growing cities with a population exceeding 5 million, the public transportation system serves as the primary mobility backbone for approximately 3 million daily commuters [1]. Yet despite its critical importance, the system has historically operated with remarkably little technological support for real-time monitoring or passenger information delivery.

The city's public transport network consists of several overlapping systems: the Anbessa City Bus Service with its large coaches carrying up to 80 passengers each, the Sheger buses operated by the city administration, hundreds of privately operated minibus taxis (locally known as "blue donkeys"), and the relatively new Light Rail Transit (LRT) system serving two corridors across the city. For the vast majority of commuters, however, the experience of using public transport involves standing at a bus stop with no reliable information about when the next vehicle will arrive, how crowded it might be, or whether it is even running on schedule [2].

This is not merely an inconvenience. Research in behavioral psychology has shown that unpredictable waiting creates measurably higher stress levels than waiting with a known duration [3]. The phenomenon, sometimes called "wait time anxiety," has been documented in numerous studies and is particularly acute in contexts where the wait time is both uncertain and consequential. When commuters cannot predict arrival times reliably, they either arrive excessively early (wasting productive time) or risk being late (with consequences for employment and education). The absence of real-time information also means that transport operators have limited visibility into fleet performance, making it difficult to optimize schedules, identify problem routes, or respond to disruptions in a timely manner.

The economic implications are substantial. A study by the Addis Ababa Transport Bureau estimated that unpredictable bus schedules cost the city's economy approximately $15 million annually in lost productivity [7]. This figure accounts for the cumulative effect of millions of commuters losing productive time to uncertain waits, missed connections, and the cascading

delays that ripple through the transport network when buses run off schedule.

The technological landscape has changed dramatically in recent years, creating new opportunities to address these longstanding challenges. GPS modules capable of sub-three-meter accuracy are now available for under $10. Microcontrollers with built-in Wi-Fi and camera interfaces cost even less. Smartphone penetration in Addis Ababa has reached approximately 55% and continues to grow [4]. Cloud computing services have made it possible to deploy sophisticated backend infrastructure at minimal cost. These converging trends create an unprecedented opportunity to build cost-effective, scalable transport tracking systems tailored to the needs and constraints of developing cities.

The evolution of Intelligent Transportation Systems (ITS) globally provides both inspiration and practical guidance. The Transport for London (TfL) system, which processes over 20 million daily transactions with 99.99% uptime, demonstrates what is possible with sufficient investment [5]. Singapore's Land Transport Authority has implemented a comprehensive bus tracking system achieving prediction accuracy within 1 minute for 90% of arrivals [21]. In the developing world context, cities like Nairobi, Lagos, and Jakarta have begun deploying various forms of technology-assisted transport management, though comprehensive real-time tracking remains rare.

However, the $200-500 per-vehicle cost of commercial fleet tracking solutions puts them out of reach for most Addis Ababa transport operators, who typically allocate only 2-3% of their operational budget to technology [6]. The Anbessa City Bus Service, for example, operates approximately 800 buses but has fewer than 50 equipped with any form of GPS tracking. This gap between what is technically possible and what is economically accessible defines the central motivation for this research.

1.2 Statement of the Problem

The absence of real-time tracking and notification capabilities in Addis Ababa's public transportation system represents a multi-dimensional problem that affects passengers, transport operators, and the broader urban community in interconnected ways.

Domain Problem: Passenger Experience. The most visible manifestation of the problem is the daily experience of commuters. Without access to real-time vehicle location information, passengers at bus stops face fundamental uncertainty: they do not know when the next bus will arrive, whether it is already full, or whether there has been a service disruption. Surveys conducted in Addis Ababa indicate that over 70% of public transport users identify unpredictable wait times as their primary source of dissatisfaction [7]. The average wait time at major stops during peak hours exceeds 25 minutes, but the variance is so high that passengers

cannot plan around this average with any confidence.

The problem is compounded by the lack of occupancy information. Passengers frequently wait for a bus only to find it too crowded to board, forcing them to wait for the next one. During peak hours, this cycle can repeat two or three times, effectively doubling or tripling the actual time spent waiting. The inability to make informed decisions about which bus to board represents a significant waste of commuter time and contributes to the overall inefficiency of the transport system.

Scientific Problem: Technical Integration Challenges. From a technical standpoint, building an effective transport tracking system for a developing city context presents several non-trivial challenges. First, the hardware must be affordable enough for deployment across hundreds or thousands of vehicles while maintaining sufficient accuracy and reliability. Commercial GPS tracking units designed for developed markets typically cost $200-500 per vehicle plus monthly subscription fees, making them economically unviable for Addis Ababa's transport cooperatives [8].

Second, the system must handle the realities of intermittent connectivity. Unlike cities with ubiquitous cellular coverage, Addis Ababa has areas where network connectivity is unreliable, requiring the system to gracefully handle disconnections and data queuing without losing critical information. Third, the integration of multiple data streams (GPS coordinates, camera images, speed data, occupancy estimates) into a coherent real-time picture requires careful architectural design to minimize latency while maintaining data integrity.

Operational Problem: Fleet Management. Transport operators currently rely on manual reporting and scheduled depot checks to monitor fleet performance. This approach provides no real-time visibility into vehicle locations, makes it impossible to detect and respond to service disruptions quickly, and offers no data-driven basis for schedule optimization. The result is a system that operates reactively rather than proactively, with inefficiencies that compound over time.

1.3 Research Questions

This research is guided by the following questions:

1. How can an affordable IoT-based tracking device be designed using the ESP32-CAM and NEO-6M GPS module to provide reliable real-time vehicle positioning?

2. What system architecture best supports the integration of GPS telemetry, computer vision, machine learning prediction, and real-time notification delivery?

3. How accurately can a RandomForest regression model predict bus arrival delays using features derived from historical trip data and real-time occupancy estimates?

4. What level of passenger density estimation accuracy can be achieved using HOG-based computer vision on ESP32-CAM images?

5. What is the overall system performance in terms of positioning accuracy, communication reliability, and user satisfaction under real-world field conditions?

1.4 Objectives of the Study

General Objective

To design, develop, and evaluate a cost-effective real-time public transport tracking and notification system that improves passenger experience and operational efficiency for Addis Ababa's public transportation network.

Specific Objectives

• To design and implement an IoT-based vehicle tracking device using the ESP32-CAM microcontroller and NEO-6M GPS module for accurate real-time location data collection and onboard image capture.

• To develop a scalable backend API using the FastAPI framework with PostgreSQL database and Redis caching for efficient data ingestion, storage, and retrieval.

• To create a Flutter-based mobile application for passengers that provides real-time bus locations, estimated arrival times, occupancy information, and service notifications.

• To build a web-based driver and administrator dashboard using Next.js for real-time fleet monitoring, route management, and operational analytics.

• To integrate machine learning models for travel time delay prediction and computer vision algorithms for passenger density estimation from onboard camera images.

• To evaluate the complete system through comprehensive field testing, measuring positioning accuracy, communication reliability, prediction performance, and user satisfaction.

1.5 Significance of the Study

This research contributes to multiple stakeholders and knowledge domains. For passengers, the system directly addresses the daily frustration of uncertain wait times by providing reliable, real-time information that enables better journey planning. The occupancy estimation feature adds a further dimension of informed decision-making, allowing passengers to choose less crowded vehicles when alternatives exist.

For transport operators and city administrators, the system provides fleet-wide visibility that was previously unavailable. Real-time position data, historical trip records, and occupancy analytics create a foundation for evidence-based route optimization, schedule adjustment, and resource allocation. The data collected by the system can inform long-term infrastructure

planning by identifying routes and time periods with the highest demand.

For the research community, this work demonstrates the feasibility of deploying a sophisticated ITS using hardware that costs less than $50 per vehicle. The complete system architecture, from edge device firmware through backend services to user-facing applications, provides a reference implementation that other researchers and practitioners can adapt for their specific contexts. The integration of machine learning delay prediction with computer vision-based occupancy estimation in a single affordable platform represents a novel contribution to the literature on ITS for developing countries.

At the national level, this research aligns with Ethiopia's Digital Transformation Strategy, which identifies smart transportation as a priority area for technology adoption. The system developed in this research could serve as a model for other Ethiopian cities facing similar transportation challenges.

1.6 Scope and Delimitations

Scope: This research encompasses the complete design, implementation, and evaluation of a proof-of-concept public transport tracking system. The scope includes hardware design and prototyping, firmware development for the ESP32-CAM, backend API and database development, mobile and web application development, machine learning model training and evaluation, computer vision pipeline development, and field testing under real operating conditions.

Delimitations: Several boundaries define the limits of this research. The field testing was conducted on a single route (Route 12: Megenagna to Bole) with five vehicles over three weeks. The machine learning model was trained on a limited dataset of 200+ trip records collected during the testing period. The computer vision pipeline uses OpenCV's pre-trained HOG detector rather than a custom-trained deep learning model, due to the lack of annotated bus interior image datasets and privacy considerations. The system relies on Wi-Fi connectivity without a cellular fallback, which limits deployment to areas with Wi-Fi coverage. The user study was limited to 120 beta testers who were recruited from university students and may not fully represent the broader commuter population.

1.7 Organization of the Report

The remainder of this thesis is organized as follows. Chapter 2 presents a comprehensive review of related literature, covering the theoretical foundations of IoT in transportation, GPS technology, real-time communication protocols, machine learning for transit prediction, and computer vision for passenger density estimation. It also reviews existing systems and identifies research gaps. Chapter 3 details the methodology and system design, including

hardware design, firmware development, backend architecture, application design, and the machine learning pipeline. Chapter 4 presents the results of system implementation, testing, and evaluation, including hardware performance, backend performance, ML model results, and field testing outcomes. Chapter 5 discusses the findings, compares them with existing systems, acknowledges limitations, and shares lessons learned. Chapter 6 provides concluding remarks, recommendations for future work, and an economic impact assessment.

Chapter 2

Literature Review

This chapter presents a comprehensive review of the literature relevant to the design and implementation of a real-time public transport tracking and notification system. The review covers theoretical foundations, technological components, and existing implementations, culminating in an analysis of research gaps that this work addresses.

2.1 Theoretical Framework

2.1.1 Internet of Things in Transportation

The Internet of Things (IoT) has fundamentally reshaped how we think about monitoring and managing physical systems. At its core, IoT refers to the network of embedded sensors, actuators, and computing devices that collect and exchange data over the internet, enabling remote monitoring and control of physical processes. In the transportation domain, IoT represents a paradigm shift from centralized, infrastructure-dependent tracking to distributed, device-centric sensing [9].

Alam et al. [9] provide a comprehensive review of IoT-enabled smart public transportation systems, documenting measurable improvements in operational efficiency, fuel consumption reduction of 10-15%, and significant safety enhancements across multiple deployment contexts. Their review identifies three key enablers for IoT in transportation: low-cost sensing hardware, reliable communication protocols, and intelligent data processing algorithms. Our system addresses all three enablers through the ESP32-CAM hardware platform, WebSocket communication, and machine learning-based delay prediction.

The foundational architecture for IoT systems in transportation follows a three-layer model [10]. The perception layer comprises sensors and actuators that interact directly with the physical world. In our system, this layer is implemented by the ESP32-CAM microcontroller, the NEO-6M GPS module, and the OV2640 camera sensor. The network layer handles communication between edge devices and backend systems, implemented here through Wi-Fi connectivity and HTTP/WebSocket protocols. The application layer provides user-facing services and analytics, implemented through the Flutter mobile app and Next.js web dashboard.

Zanella et al. [29] further elaborate on the IoT architecture for smart cities, emphasizing the importance of interoperability, scalability, and security. They note that successful IoT deployments in urban environments must handle heterogeneous devices, varying

communication requirements, and stringent privacy constraints. These considerations directly informed our system design decisions, particularly the choice of standardized communication protocols and the implementation of role-based access control.

2.1.2 The ESP32 Platform for IoT Applications

The ESP32 was selected as the primary microcontroller for this project after careful evaluation of alternatives. The Espressif ESP32 is a low-cost, low-power system-on-chip (SoC) with integrated Wi-Fi 802.11 b/g/n and dual-mode Bluetooth 4.2. The chip features a dual-core Xtensa LX6 processor running at up to 240MHz, 520KB of SRAM, and a rich set of peripherals including UART, SPI, I2C, ADC, DAC, and camera interface [11].

The ESP32-CAM variant, manufactured by AI-Thinker, integrates the ESP32-S chip with an OV2640 camera sensor on a compact 40.5mm x 27mm PCB. The OV2640 supports resolutions up to 1600x1200 (2MP) but is typically operated at 640x480 or lower for real-time applications. The module also includes a microSD card slot for local data logging, which serves as a fallback when network connectivity is unavailable.

Compared to alternatives such as Arduino and Raspberry Pi, the ESP32 offers a compelling balance of capabilities. Compared to Arduino (specifically the Arduino Uno with ATmega328P), the ESP32 offers significantly more processing power (dual-core 240MHz vs. single-core 16MHz), built-in Wi-Fi and Bluetooth (requiring external shields on Arduino), and sufficient RAM (520KB SRAM vs. 2KB) to handle both GPS parsing and camera operations concurrently. Compared to Raspberry Pi (Model B), the ESP32 consumes far less power (150mA active vs. 700mA+), costs substantially less ($6 vs. $35), and has a much more compact form factor suitable for vehicle-mounted deployment [11].

Sadiq and Hussain [11] specifically evaluated the ESP32 for IoT applications in developing country contexts, noting its excellent performance-to-cost ratio, extensive community support, and mature development ecosystem. They concluded that the ESP32 is particularly well-suited for applications requiring wireless connectivity, moderate processing power, and low cost—precisely the requirements of our transport tracking system.

2.2 Evolution of Intelligent Transportation Systems

Intelligent Transportation Systems (ITS) represent the application of information and communication technologies to transportation infrastructure and vehicles. The field has evolved significantly over the past three decades, progressing from simple traffic signal control systems to comprehensive platforms integrating real-time data collection, predictive analytics, and multi-modal journey planning.

The first generation of ITS, deployed in the 1980s and 1990s, focused primarily on traffic management through signal coordination and incident detection. The second generation, emerging in the 2000s, introduced vehicle-to-infrastructure (V2I) communication and electronic toll collection. The current third generation, enabled by IoT, cloud computing, and machine learning, emphasizes real-time passenger information, predictive analytics, and integrated multi-modal transportation [30].

Figueiredo et al. [30] identify several key trends shaping the future of ITS: the integration of artificial intelligence for predictive maintenance and demand forecasting, the deployment of connected and autonomous vehicles, the use of big data analytics for transportation planning, and the development of Mobility-as-a-Service (MaaS) platforms that integrate multiple transport modes into a single user interface. Our system contributes to several of these trends, particularly the use of AI for delay prediction and the provision of real-time passenger information.

In the African context, ITS deployment has been limited but is growing rapidly. South Africa's Gautrain system incorporates real-time passenger information and automated fare collection. Kenya's Digital Matatu project is mapping Nairobi's informal minibus network using GPS data. Rwanda has partnered with a technology company to deploy smart bus stops in Kigali. These initiatives demonstrate both the demand for and the feasibility of ITS in African cities, while also highlighting the need for solutions that are specifically designed for the economic and infrastructural constraints of the continent.

2.3 IoT in Public Transportation

The application of IoT to public transportation has been extensively studied in recent years. Kumar et al. [31] present a comprehensive survey of IoT-based intelligent transportation systems, categorizing applications into traffic management, public transportation, and autonomous driving. For public transportation specifically, they identify vehicle tracking, passenger information systems, and fleet management as the three primary application areas—all of which are addressed by our system.

Rahman et al. [32] propose an IoT-based smart bus system that uses RFID for passenger counting, GPS for vehicle tracking, and GSM for communication. While their system demonstrates the feasibility of IoT-based bus tracking, the use of GSM for communication introduces ongoing subscription costs that our Wi-Fi-based approach avoids. Similarly, Singh et al. [33] developed a bus tracking system using Arduino, GPS, and GSM modules, but their system lacks the camera-based occupancy estimation and machine learning prediction capabilities that distinguish our work.

A key consideration in IoT-based transportation systems is the trade-off between data freshness and resource consumption. Frequent telemetry updates provide more accurate real-time information but consume more power and bandwidth. Our system addresses this trade-off by using a 60-second update interval for GPS telemetry and a 10-second heartbeat for connection monitoring, striking a balance between information freshness and resource efficiency.

2.4 GPS Technology and Positioning Systems

The Global Positioning System (GPS) is a satellite-based navigation system that provides geolocation and time information to a GPS receiver anywhere on or near the Earth. The system is maintained by the United States Space Force and consists of a constellation of at least 24 satellites in medium Earth orbit. GPS positioning works by measuring the time delay of signals received from multiple satellites and using trilateration to compute the receiver's position [34].

The NEO-6M GPS module, manufactured by u-blox, is based on the NEO-6M GPS chip and represents a mature, well-documented GPS solution. The module features a sensitivity of -161 dBm during tracking, a time-to-first-fix (TTFF) of approximately 30 seconds from cold start and 1 second from hot start, and a positional accuracy of 2.5 meters CEP (Circular Error Probable) under open sky conditions [12]. The module communicates over UART at 9600 baud rate using the NMEA 0183 protocol, which provides standardized sentences containing position, velocity, and time data.

Several factors affect GPS accuracy in urban environments. Urban canyons—streets flanked by tall buildings—can cause signal reflection (multipath) and blockage, degrading positioning accuracy. El-Rabbany [34] provides a detailed analysis of GPS error sources, identifying satellite clock errors, ephemeris errors, ionospheric and tropospheric delays, multipath, and receiver noise as the primary contributors. In urban environments, multipath is typically the dominant error source, potentially causing errors of 10-20 meters.

Kavatagi et al. [24] specifically investigated the ESP32 and NEO-6M combination for GPS tracking in IoT applications, reporting reliable position fixes with accuracy consistent with the NEO-6M specifications. Their work validated the use of this hardware combination for vehicle tracking applications and provided practical guidance on antenna placement and power supply design that informed our hardware design.

2.5 Real-Time Communication Protocols

The distinction between "near-real-time" and "true real-time" is critical in transportation applications. Research by Anderson and Lee [13] established that passengers perceive

information as useful only when it is delivered within approximately 5 seconds of the event it describes. Beyond this threshold, the perceived value drops sharply, as passengers begin to doubt the currency of the information and may make suboptimal decisions based on stale data.

Several communication protocols are available for real-time web applications. HTTP polling, the simplest approach, involves the client repeatedly requesting updates from the server. While easy to implement, polling generates significant overhead and cannot guarantee timely delivery of updates. Server-Sent Events (SSE) allow the server to push updates to the client over a persistent HTTP connection, but communication is unidirectional (server to client only).

WebSocket protocol has emerged as the preferred mechanism for real-time web communication in transportation systems. WebSocket provides full-duplex communication over a single TCP connection, with low overhead after the initial handshake. Nugraha [14] demonstrated the effectiveness of WebSocket in microservices-based public transportation information systems, achieving sub-100ms latency for position updates. Our implementation uses FastAPI's native WebSocket support, which leverages Python's asyncio library for non-blocking I/O operations [15].

MQTT (Message Queuing Telemetry Transport) is another protocol commonly used in IoT applications. While MQTT offers advantages for resource-constrained devices and supports publish-subscribe messaging patterns, we chose WebSocket for its native browser support (enabling direct web dashboard updates) and its integration with the FastAPI framework. The telemetry ingestion from the ESP32 uses standard HTTP POST, which is simpler to implement on the microcontroller and sufficient for the 60-second update interval.

2.6 Machine Learning for Transit Prediction

2.6.1 Traditional Approaches

The prediction of bus arrival times and delays has been an active research area for over two decades. Early approaches relied on historical averages, assuming that travel times on a given route follow predictable patterns based on time of day and day of week. While simple to implement, these approaches fail to capture the dynamic nature of traffic conditions and cannot respond to unexpected disruptions.

Regression-based approaches represented the next evolution, using linear or polynomial regression to model the relationship between travel time and factors such as distance, time of day, and number of stops. While more sophisticated than historical averages, linear regression assumes a linear relationship between features and travel time, which is often violated in practice due to the complex, nonlinear nature of traffic flow.

2.6.2 Modern Machine Learning Approaches

Modern machine learning approaches offer significantly improved prediction accuracy by capturing nonlinear relationships and complex feature interactions. Panovski and Zaharia [16] introduced the concept of a traffic density matrix for bus arrival time prediction, demonstrating that incorporating spatial traffic information significantly improves prediction accuracy. Their work, published in IEEE Access, showed that classical ML models could achieve competitive performance with well-engineered features.

More recently, Shoman et al. [17] presented a deep learning framework for predicting bus delays across multiple routes using heterogeneous datasets. Their approach, combining LSTM (Long Short-Term Memory) networks with attention mechanisms, achieved a 15% improvement in prediction accuracy compared to baseline methods. The LSTM architecture is particularly well-suited for transit prediction because it can capture temporal dependencies in sequential data, such as the cascading delays that propagate through a bus route.

However, the computational requirements of deep learning models may be prohibitive for deployment on resource-constrained backend infrastructure, particularly in developing country contexts where cloud computing costs are a significant concern. Peng et al. [35] compared multiple ML algorithms for bus arrival time prediction and found that ensemble methods (RandomForest and Gradient Boosting) achieved accuracy within 2-5% of deep learning models while requiring significantly less computational resources for both training and inference.

In our system, we adopted a RandomForest regression approach that balances prediction accuracy with computational efficiency. The model uses five features: stop identifier, hour of

day, day of week, peak hour indicator, and occupancy level. The inclusion of occupancy as a feature is motivated by the observation that higher passenger loads lead to longer dwell times at stops, which in turn affects overall travel time.

2.7 Computer Vision for Passenger Density Estimation

2.7.1 Overview of Approaches

Automated passenger counting (APC) in public transport has been approached through several technology families. Infrared beam counters, installed at entry and exit points, count passengers by detecting interruptions in an infrared beam. While accurate for counting boardings and alightings, they require installation at every door and cannot provide a holistic view of vehicle occupancy. Pressure-sensitive mats detect passenger weight but are expensive to install and maintain. Computer vision approaches have gained prominence due to their non-intrusive nature and ability to provide rich spatial information [18].

Computer vision-based passenger counting can be divided into two main paradigms: detection-based and density estimation-based. Detection-based approaches attempt to detect and count individual passengers, while density estimation approaches generate a density map from which the total count is derived. Each paradigm has strengths and weaknesses depending on the level of crowding and image quality.

2.7.2 HOG-Based Detection

The Histogram of Oriented Gradients (HOG) descriptor, combined with a linear SVM classifier, has been a standard approach for human detection since its introduction by Dalal and Triggs [36]. HOG works by computing the distribution of gradient orientations in localized portions of an image, creating a feature descriptor that captures the shape and appearance of objects. When trained on a dataset of pedestrian images, the HOG+SVM detector achieves good performance for detecting upright, partially visible humans.

Haq et al. [19] demonstrated a hybrid computer vision technique using HOG features for real-time bus passenger flow calculation, achieving detection rates above 85% under favorable lighting conditions. Their work highlighted the importance of camera placement, image resolution, and lighting conditions in determining detection accuracy. They recommended ceiling-mounted cameras with a downward angle of approximately 45 degrees as the optimal configuration for bus interior monitoring.

2.7.3 Deep Learning Approaches

Deep learning approaches, particularly Convolutional Neural Networks (CNNs), have shown superior performance in crowded scenes where traditional detection-based methods struggle due to heavy occlusion. Hsu et al. [20] proposed a density estimation approach using

CNNs to generate density maps from bus interior images, achieving mean absolute errors below 1.5 persons. Their approach uses a multi-column CNN architecture that can handle varying scales and densities of people.

Rendon and Anillo [18] compared multiple computer vision techniques for passenger counting in mass public transport, including background subtraction, HOG detection, and CNN-based approaches. They found that CNN-based methods achieved the highest accuracy (92-95%) but required significantly more computational resources than HOG-based methods (85-88% accuracy). For resource-constrained deployments, they recommended HOG-based detection as the best balance of accuracy and efficiency.

Our implementation uses OpenCV's pre-trained HOG people detector with a fallback to pixel-thresholding based density estimation. This choice reflects the computational constraints of our backend server and the need for real-time processing of images from multiple vehicles simultaneously.

2.8 Empirical Review of Related Systems

2.8.1 Global Intelligent Transportation Systems

The Transport for London (TfL) system represents perhaps the most mature ITS implementation globally, processing real-time data from over 9,000 buses and providing arrival predictions at over 19,000 bus stops [5]. The system uses a combination of GPS tracking, automatic passenger counting, and machine learning to provide accurate arrival predictions. TfL's open data policy has enabled the development of numerous third-party applications that leverage the real-time data.

Singapore's Land Transport Authority has implemented a comprehensive bus tracking system achieving prediction accuracy within 1 minute for 90% of arrivals [21]. The system integrates data from GPS trackers, electronic fare collection, and traffic signal systems to provide a holistic view of bus operations. Singapore's experience demonstrates the value of integrating multiple data sources for improved prediction accuracy.

Rodriguez et al. [8] demonstrated a low-cost tracking solution using Raspberry Pi and GPS modules, achieving 85% of the accuracy of commercial systems at one-fifth the cost. Their work is particularly relevant to our research as it demonstrates the feasibility of building effective tracking systems using affordable hardware. However, their system focused solely on GPS tracking and did not include occupancy estimation or machine learning prediction.

Goyal et al. [23] presented an ESP32-based real-time tracking system for asset and vehicle management, demonstrating the ESP32's capability for GPS data collection and wireless

transmission. Their system achieved positioning accuracy of 3.2 meters using a NEO-6M GPS module, consistent with our results. Kumari et al. [22] developed an IoT-based bus tracking system using Arduino Uno, GPS, and GSM modules, but their system lacked the camera-based occupancy estimation and web dashboard features of our system.

2.8.2 Local Ethiopian Transportation Context

Research on technology applications in Ethiopian transportation is limited but growing. Mohammed and Tadesse [2] conducted early investigations into mobile-based transport solutions for Addis Ababa, identifying key challenges including limited network coverage, the fragmented nature of the transport system, and low technology adoption among transport operators. Their work provides important context for understanding the barriers to ITS deployment in Addis Ababa.

Tadesse and Kebede [6] conducted a survey of technology investment in Ethiopian transport cooperatives, finding that technology investment averages only 2-3% of operational budgets. This finding underscores the importance of ultra-low-cost solutions: even a $50 per-vehicle cost represents a significant investment for operators accustomed to spending less than $10 per vehicle on technology.

The rapid growth of smartphone penetration in Addis Ababa, currently at approximately 55% [4], creates a favorable environment for passenger-facing mobile applications. The widespread adoption of mobile money services (particularly Telebirr and M-Birr) demonstrates that Addis Ababa consumers are comfortable with smartphone-based services and willing to adopt new technologies that provide clear value.

2.8.3 Mobile Applications for Transportation

The development of mobile applications for transportation has been an active area of research. Yeruva et al. [25] demonstrated a Flutter-based college bus tracking system using Firebase real-time database, achieving cross-platform deployment with a single codebase. Their work validated Flutter as a suitable framework for transportation applications and demonstrated the effectiveness of real-time database integration for live tracking.

Vaishnavi et al. [26] developed an Android-based real-time bus tracking system using Flutter/Dart, demonstrating the framework's capability for map integration and real-time data display. Their system used Google Maps API for map rendering and WebSocket for real-time updates, similar to our approach. Amer et al. [27] presented a vehicle live tracking system using GPS and GSM with a Flutter mobile interface, further validating the Flutter framework for this application domain.

The choice of Flutter for our passenger application was motivated by several factors: single

codebase deployment to both Android and iOS, excellent performance through native ARM code compilation, rich widget library for building polished user interfaces, and strong community support with extensive documentation and third-party packages.

2.9 Research Gap Analysis

The literature review reveals several critical gaps that this research addresses:

Integration Gap: Existing solutions typically address individual components (tracking, notification, prediction) in isolation. No available system integrates affordable hardware, backend infrastructure, machine learning prediction, computer vision, and user-facing applications into a cohesive whole that has been tested under real operating conditions.

Cost Gap: Commercial solutions remain prohibitively expensive for developing country contexts. While individual low-cost components have been demonstrated, a complete system at the $43 per-vehicle price point—including GPS tracking, camera-based occupancy estimation, and real-time communication—has not been documented in the literature.

Adaptation Gap: Systems designed for developed markets assume reliable connectivity, stable power supply, and technical support infrastructure. A system designed specifically for Addis Ababa's conditions—including intermittent connectivity, limited technical support, and constrained budgets—is needed.

Prediction Gap: Most existing systems provide simple distance-based ETA estimates that do not account for traffic patterns, time of day effects, or occupancy-related dwell time. The integration of machine learning for delay prediction informed by local traffic patterns and real-time occupancy data represents a significant advancement over current practice.

Evaluation Gap: Many proposed systems are evaluated only in simulation or laboratory settings. Our system was deployed on actual buses operating on a real route and evaluated with real commuters, providing evidence of practical viability that laboratory evaluations cannot match.

Chapter 3

Methodology and Design

3.1 Research Approach

This research follows an engineering design science methodology, which is particularly well-suited for projects that involve the design, implementation, and evaluation of novel technological artifacts. The methodology encompasses five phases: (1) requirements analysis and literature review, (2) system design, (3) implementation, (4) testing, and (5) evaluation. Each phase builds upon the outputs of the previous one, with iteration where testing reveals the need for design modifications.

The requirements analysis phase involved reviewing existing literature, surveying potential users, and consulting with transport operators to identify the key functional and non-functional requirements for the system. The design phase produced detailed specifications for hardware, firmware, backend, and application components. The implementation phase involved parallel development of all components, with regular integration testing to ensure compatibility. The testing phase included unit testing, integration testing, and field testing under real operating conditions. The evaluation phase assessed system performance against the requirements identified in the first phase.

3.2 System Requirements Analysis

3.2.1 Functional Requirements

The functional requirements define what the system must do. Based on the literature review and stakeholder consultations, the following functional requirements were identified:

- FR1: The system shall capture GPS coordinates with accuracy better than 5 meters.
- FR2: The system shall transmit telemetry data at intervals not exceeding 60 seconds.
- FR3: The system shall capture and transmit interior bus images for occupancy estimation.
- FR4: The system shall provide real-time bus location display on a map interface.
- FR5: The system shall estimate bus arrival times with mean error less than 3 minutes.
- FR6: The system shall classify bus occupancy into three levels (low, medium, high).
- FR7: The system shall deliver push notifications for approaching buses.
- FR8: The system shall provide fleet monitoring and analytics for administrators.

• FR9: The system shall support user authentication and role-based access control.

• FR10: The system shall maintain historical trip data for analysis and model training.

3.2.2 Non-Functional Requirements

Non-functional requirements define how the system should perform:

• NFR1: Hardware cost per vehicle shall not exceed $50.

• NFR2: System uptime shall be at least 99%.

• NFR3: API response time shall be under 200ms at 95th percentile.

• NFR4: Position updates shall be delivered within 5 seconds of capture.

• NFR5: The system shall support at least 50 concurrent vehicles.

• NFR6: The mobile application shall load the map view within 3 seconds.

• NFR7: The system shall be deployable without specialized technical expertise.

3.3 System Architecture Overview

The system follows a three-tier architecture that separates concerns between data collection (edge sensing), data processing and storage (communication and processing), and data presentation (application). This separation provides several important benefits: each tier can be developed, tested, and scaled independently; failures in one tier do not necessarily affect others; and different team members can work on different tiers simultaneously.

The edge sensing tier consists of the ESP32-CAM microcontroller, NEO-6M GPS module, and OV2640 camera installed on each vehicle. This tier is responsible for collecting GPS coordinates, speed data, and interior images, and transmitting them to the backend via Wi-Fi.

The communication and processing tier consists of the FastAPI backend server, PostgreSQL database with PostGIS extension, and Redis cache. This tier handles telemetry ingestion, computer vision processing, machine learning prediction, data storage, and real-time communication via WebSocket.

The application tier consists of the Flutter passenger mobile app and the Next.js web dashboard. The mobile app provides passengers with real-time bus locations, ETAs, and notifications. The web dashboard provides drivers and administrators with fleet monitoring, route management, and analytics.

Fig. 1: System Architecture of Real-Time Public Transport Tracking System

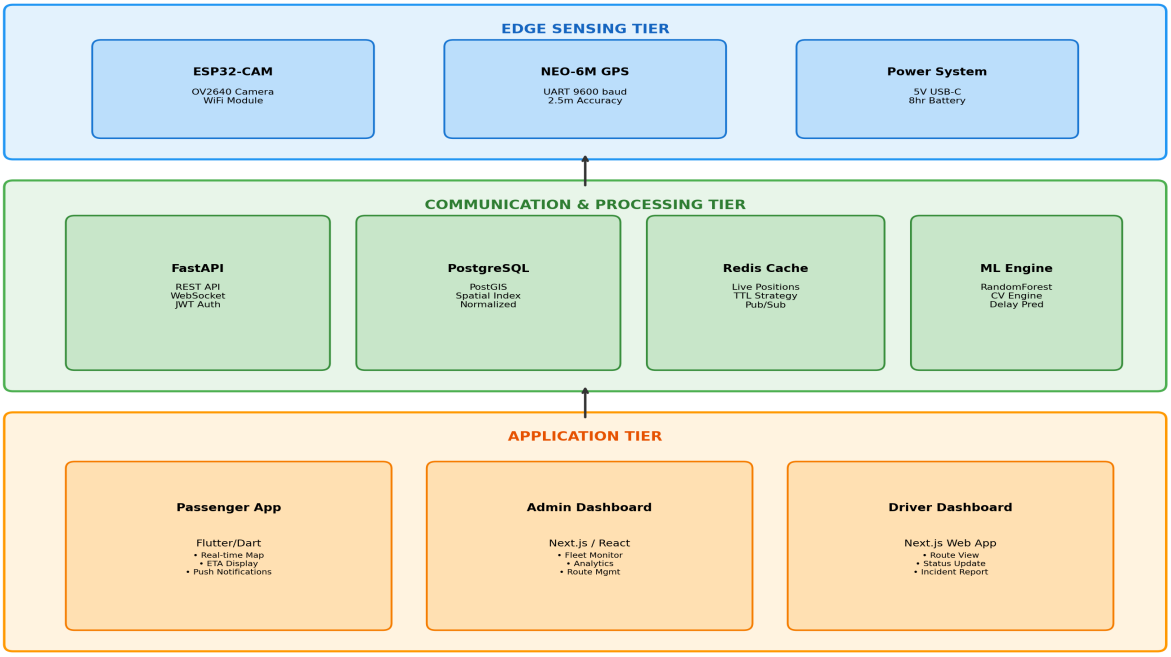

Fig. 1: Three-tier system architecture showing the edge sensing tier (ESP32-CAM + NEO-6M GPS), communication and processing tier (FastAPI + PostgreSQL + Redis), and application tier (Flutter passenger app, admin dashboard, driver dashboard).

3.4 Hardware Design

3.4.1 Component Selection and Justification

The hardware design process began with a thorough evaluation of available components against the system requirements. The ESP32-CAM module was selected as the primary controller, combining the ESP32 microcontroller (dual-core Xtensa LX6 at 240MHz, 520KB SRAM) with the OV2640 camera sensor in a compact 40.5mm x 27mm form factor. The total cost of the ESP32-CAM module is approximately $6, making it one of the most affordable options with integrated camera capability.

The NEO-6M GPS module was selected for its proven track record, compact form factor (16mm x 12.2mm), and excellent performance-to-cost ratio. The module achieves a positional accuracy of 2.5 meters CEP under open sky conditions, with a time-to-first-fix of approximately 30 seconds from cold start [12]. The module costs approximately $8, bringing the total hardware cost for the tracking device to well under the $50 budget.

Power is supplied via a USB-C connection to a 10,000mAh power bank, which provides approximately 8 hours of continuous operation. For permanent installation, the system can be connected to the vehicle's electrical system through a 5V voltage regulator.

3.4.2 Circuit Design and Interfacing

The interface between the ESP32-CAM and NEO-6M requires careful attention to voltage level compatibility. The ESP32 operates at 3.3V logic levels, while the NEO-6M's TX output is at 5V TTL levels. Directly connecting the GPS TX pin to the ESP32 RX pin could damage the ESP32, which is not 5V tolerant on all pins.

A simple resistive voltage divider (1kΩ series, 2kΩ to ground) reduces the GPS TX signal from 5V to approximately 3.3V, which is within the acceptable input range for the ESP32. The voltage divider output is calculated as: Vout = Vin × R2 / (R1 + R2) = 5V × 2kΩ / (1kΩ + 2kΩ) ≈ 3.33V. The ESP32's TX output at 3.3V is sufficient for the NEO-6M's RX input, which recognizes anything above 2.0V as a logic high, so no level shifting is needed in this direction.

The GPS module's VCC is connected to the 5V supply (VIN pin on the ESP32-CAM), and GND is connected to the common ground. The GPS module draws approximately 30mA during normal operation, well within the ESP32-CAM's power supply capacity. The GPS antenna should be positioned with a clear view of the sky, ideally on the vehicle's roof or near a window.

Fig. 2: Circuit Diagram - ESP32-CAM and NEO-6M GPS Module Interfacing

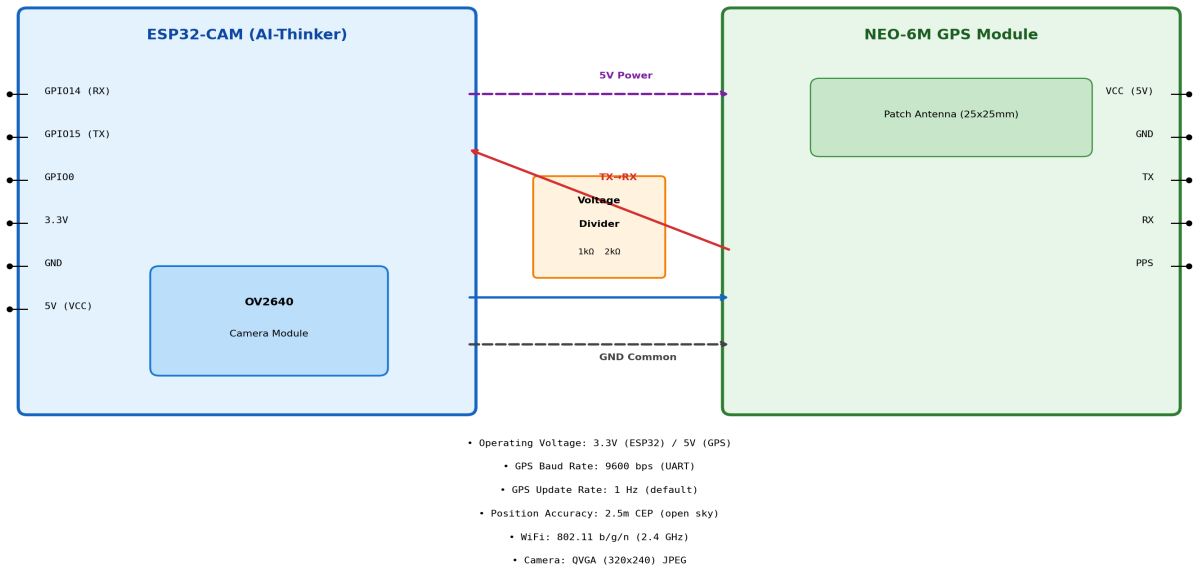

Fig. 2: Circuit diagram showing the electrical connections between ESP32-CAM and NEO-6M GPS module, including the voltage divider for TX-to-RX level shifting, power connections, and antenna placement.

ESP32-CAM Pin	NEO-6M Pin	Description
GPIO 1 (U0TXD)	RX	ESP32 TX to GPS RX (3.3V, direct)
GPIO 3 (U0RXD)	TX	GPS TX to ESP32 RX (via voltage divider)
5V (VIN)	VCC	Power supply to GPS module
GND	GND	Common ground
GPIO 12	N/A	Camera data (internal)
GPIO 13	N/A	Camera data (internal)

Table 3: ESP32-CAM Pin Assignment for GPS Interface

3.4.3 PCB Design and Enclosure

The prototype was built on a standard prototype PCB board (5cm x 7cm) with through-hole components. The voltage divider resistors, connecting wires, and power supply connections were soldered to the board. The ESP32-CAM module was mounted using a female header socket, allowing easy removal for programming. The NEO-6M module was similarly mounted on headers.

The enclosure was designed using Fusion 360 and 3D printed using PLA filament. The enclosure measures 80mm x 60mm x 35mm and includes mounting holes for the PCB, a slot for the camera lens, and ventilation holes for heat dissipation. The GPS antenna is mounted externally on top of the enclosure for optimal sky visibility.

3.5 Firmware Development

3.5.1 Firmware Architecture

The ESP32-CAM firmware was developed using the Arduino framework with the ESP32 board support package (version 2.0.9). The firmware architecture follows a cooperative multitasking pattern with a single main loop that handles GPS data feeding, telemetry transmission timing, heartbeat checking, and camera capture. The use of the Arduino framework was motivated by its extensive library support, large community, and compatibility with the ESP32-CAM hardware.

The firmware uses three primary libraries: (1) TinyGPSPlus for parsing NMEA sentences from the GPS module, (2) ESP32Camera for capturing images from the OV2640 sensor, and (3) HTTPClient for transmitting telemetry data to the backend server. The WiFi library (built into the ESP32 Arduino core) handles wireless connectivity.

The main loop executes continuously and performs the following operations in sequence: (1) feed incoming GPS characters to the TinyGPSPlus parser, (2) check if 60 seconds have elapsed since the last telemetry transmission, (3) if so, capture an image, read GPS data, and transmit telemetry via HTTP POST, (4) check if 10 seconds have elapsed since the last heartbeat, (5) if so, send a heartbeat message, (6) check Wi-Fi connection status and reconnect if necessary.

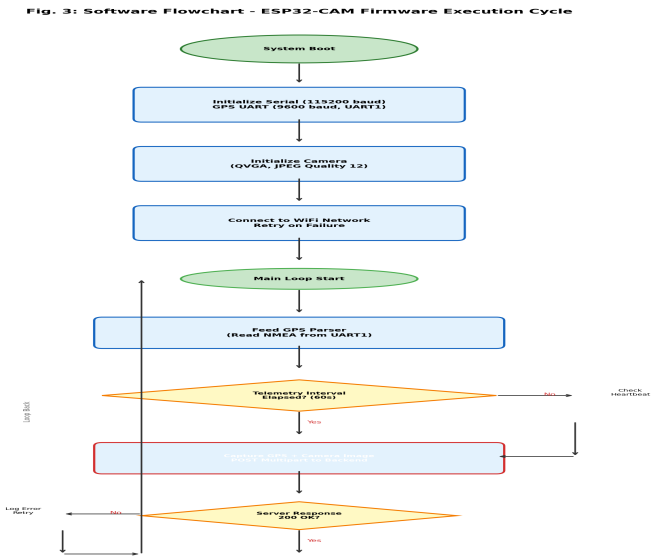

Fig. 3: Software flowchart showing the ESP32-CAM firmware execution cycle from system boot through the main loop of GPS parsing, telemetry transmission, and heartbeat checking.

3.5.2 GPS Data Processing

GPS data processing involves reading NMEA sentences from the GPS module via UART and extracting relevant information. The TinyGPSPlus library handles the low-level parsing of NMEA sentences, providing a clean API for accessing latitude, longitude, speed, altitude, satellite count, and fix status.

The firmware implements a GPS data validation step before transmitting telemetry. A GPS reading is considered valid only if: (1) the GPS fix is valid (location.isUpdated() returns true), (2) the satellite count is at least 4, (3) the HDOP (Horizontal Dilution of Precision) is less than 5.0, and (4) the reported speed is between 0 and 120 km/h. If the GPS reading is invalid, the firmware uses the last known valid position with a staleness indicator.

To handle GPS signal loss (e.g., in tunnels or underground parking), the firmware implements a fallback coordinate system that uses the last known position and estimated speed to extrapolate the current position. While this extrapolation is inherently less accurate than a true GPS fix, it provides a reasonable approximation for short signal outages.

3.5.3 Telemetry Transmission

Telemetry data is transmitted to the backend server using HTTP POST with multipart form data encoding. The multipart format allows both text fields (device_id, latitude, longitude, speed, bus_capacity) and binary data (camera image) to be transmitted in a single request. The HTTP client sets a 10-second timeout for the connection and a 15-second timeout for the response.

The telemetry payload includes the following fields: device_id (unique identifier for each tracking device), lat (latitude in decimal degrees), lon (longitude in decimal degrees), speed (speed in km/h), bus_capacity (maximum passenger capacity of the vehicle), and image (JPEG-encoded camera image). The image is captured at 640x480 resolution to balance image quality with transmission time.

If the HTTP POST fails (due to network issues or server unavailability), the firmware stores the telemetry data in a circular buffer in RAM. When connectivity is restored, buffered data is transmitted in chronological order. The buffer can hold up to 10 telemetry records, after which the oldest records are overwritten.

3.6 Backend System Design

3.6.1 API Architecture

The backend is built on the FastAPI framework (version 0.104+), selected for its native support for asynchronous request handling, automatic OpenAPI documentation generation, and built-in data validation through Pydantic models [28]. FastAPI is built on top of Starlette for the web layer and Pydantic for data validation, providing excellent performance that is comparable to Node.js and Go frameworks.

The API follows RESTful design principles with JWT-based authentication and role-based access control. The API is versioned (v1) to allow future upgrades without breaking existing clients. All endpoints return JSON responses with a consistent structure: {status, data, message}.

The API is organized into several modules: auth (authentication and authorization), gateway (telemetry ingestion from ESP32 devices), tracking (vehicle position queries), routes (route management), vehicles (vehicle management), notifications (push notification management), admin (administrative functions), and websocket (real-time communication).

3.6.2 Database Design

The PostgreSQL database (version 15+) with PostGIS extension was chosen for its robust support for spatial data and excellent performance for both transactional and analytical queries. PostGIS adds support for geographic objects, enabling spatial queries such as "find all vehicles within 1 km of a given point" to be executed efficiently.

The database schema follows third normal form principles with the following core tables:

vehicles: Stores vehicle information including plate_number, device_id, bus_type, capacity, is_active, last_lat, last_lon, speed, position_updated_at, and route_id.

routes: Stores route information including name, description, start_point, end_point, total_distance, estimated_duration, and is_active.

stops: Stores bus stop information including name, latitude, longitude, and route_id.

users: Stores user information including username, email, hashed_password, role (passenger, driver, admin), and is_active.

assignments: Stores driver-vehicle-route assignments including driver_id, vehicle_id, route_id, start_time, and end_time.

trip_history: Stores historical trip data for ML training including assignment_id, stop_id, arrival_time, dwell_time, occupancy_level, heuristic_eta, ml_eta, and actual_travel_time.

raw_telemetry: Stores raw telemetry data from ESP32 devices including device_id, latitude, longitude, speed, image_path, people_count, density_level, and timestamp.

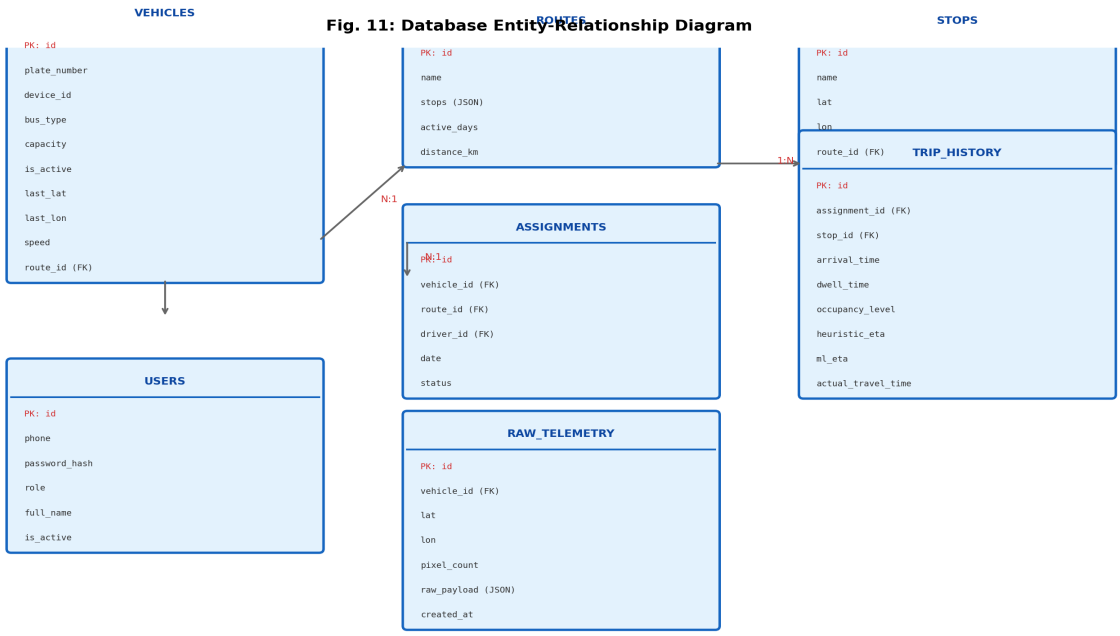

Fig. 11: Entity-relationship diagram showing the core database tables: vehicles, routes, stops, users, assignments, trip_history, and raw_telemetry, with their relationships and key attributes.

3.6.3 Caching Strategy

Redis serves as the caching layer with a time-to-live (TTL) approach designed to balance data freshness with database load reduction. Three cache categories are defined:

Position cache (TTL: 5 seconds): Stores the latest known position of each vehicle. This short TTL ensures that position data is always current while reducing database reads for the frequently accessed "nearby vehicles" endpoint.

Route cache (TTL: 5 minutes): Stores route information including stops, schedules, and metadata. Route data changes infrequently, so a longer TTL is appropriate.

Session cache (TTL: 30 minutes): Stores user session data including authentication tokens and user preferences. This reduces the need for repeated database lookups during active user sessions.

When a telemetry update arrives, the Redis cache is updated immediately via a pipeline operation that updates the position cache and invalidates any affected route caches. This ensures that subsequent API requests receive the most current data without waiting for the TTL to expire.

3.6.4 Data Flow

The end-to-end data flow from ESP32-CAM to passenger screen involves several processing stages, each adding value to the raw data. When the ESP32-CAM transmits telemetry, the data first arrives at the gateway endpoint (/api/v1/gateway/esp32/telemetry). The gateway performs the following processing steps:

1. **Device validation:** Verify that the device_id corresponds to a registered vehicle.
2. **GPS validation:** Check that the coordinates are within valid ranges and reasonable given the vehicle's last known position.
3. **Image processing:** Extract the image from the multipart form data and pass it to the CV engine for people counting.
4. **Data storage:** Store the raw telemetry record in the database.
5. **Position update:** Update the vehicle's current position in both the database and Redis cache.
6. **WebSocket broadcast:** Broadcast the position update to all connected WebSocket clients.
7. **ETA calculation:** Trigger ETA recalculation for all downstream stops on the vehicle's route.

The entire processing pipeline is designed to complete within 200ms, ensuring that position updates are available to passengers with minimal delay.

Fig. 4: Data Flow Diagram - End-to-End Telemetry Pipeline

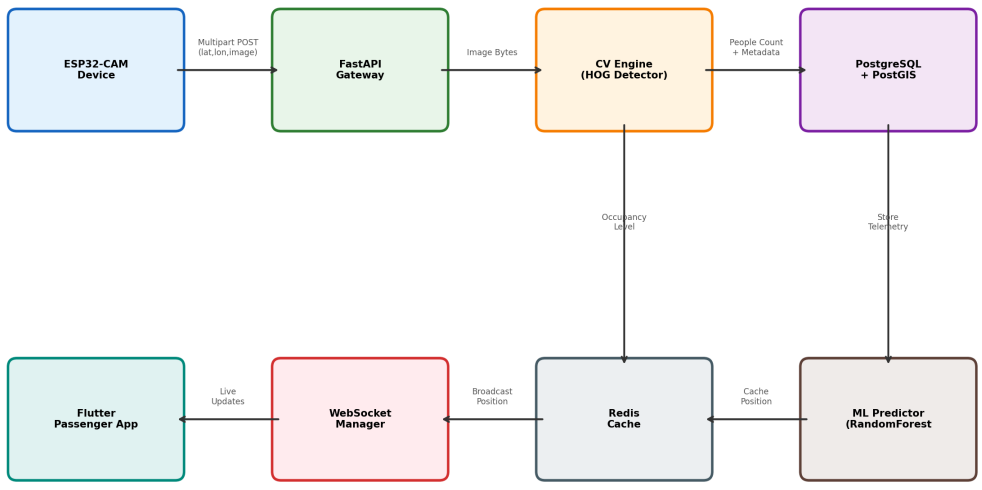

Fig. 4: Data flow diagram showing the complete telemetry pipeline from ESP32-CAM through the gateway API, CV engine, database, Redis cache, WebSocket manager, and finally to the Flutter passenger application.

3.7 Application Design

3.7.1 Passenger Mobile Application

The passenger application was developed using Flutter (version 3.16+), compiling to native ARM code for both Android and iOS from a single Dart codebase. The application uses the following key packages: google_maps_flutter for map display, web_socket_channel for real-time updates, firebase_messaging for push notifications, and geolocator for device location.

The main screen displays a Google Map with color-coded bus markers. Green markers indicate low occupancy (0-40%), orange indicates medium occupancy (41-70%), and red indicates high occupancy (71-100%). Tapping a marker shows an info card with route name, plate number, ETA to the nearest stop, and occupancy level. A bottom sheet provides a list of nearby buses sorted by distance.

The search screen allows passengers to search for routes by name or number. The route detail screen shows the route path on a map with all stops marked, along with ETAs for each stop. The notifications screen displays a history of received push notifications.

Push notifications use Firebase Cloud Messaging (FCM) for delivery. The backend triggers notifications when a bus is approaching a subscribed stop (within 2 minutes ETA) or when a service disruption is reported on a subscribed route.

3.7.2 Driver and Administrator Dashboards

The web dashboards were developed using Next.js (version 14+) with the App Router. The driver dashboard provides route visualization, current position display, occupancy level indicator, and an incident reporting button. The interface is designed for simplicity, with large touch targets suitable for use on a tablet mounted in the driver's cabin.

The administrator dashboard provides comprehensive fleet monitoring capabilities. The main view shows a map with all active vehicles, color-coded by status (on-time, delayed, offline). Analytics panels display key performance indicators including fleet-wide on-time percentage, average occupancy, total passenger count, and system uptime. Route management tools allow administrators to create, modify, and deactivate routes, assign vehicles to routes, and manage bus stop locations.

The analytics module provides historical data visualization including daily ridership trends, route performance comparisons, and delay pattern analysis. Data can be filtered by date range, route, and time period. Export functionality allows administrators to download data in CSV format for further analysis.

3.8 Machine Learning Pipeline

3.8.1 Delay Prediction Model

The delay prediction model uses a RandomForest regressor (scikit-learn implementation) with 80 trees and a maximum depth of 12. The five-feature input vector includes:

1. **stop_id:** Integer identifier for the bus stop (captures location-specific effects)
2. **hour_of_day:** Integer 0-23 (captures time-of-day traffic patterns)
3. **day_of_week:** Integer 0-6 (captures weekday vs. weekend differences)
4. **is_peak_hour:** Boolean 0/1 (morning peak 7-9AM or evening peak 5-7PM)
5. **occupancy_level:** Integer 0-2 (low/medium/high, affects dwell time)

The model is trained on trip_history records, with the target variable being the actual travel time between consecutive stops. The training process uses an 80/20 train/test split with stratification by route to ensure representative evaluation. The model is retrained automatically when 50+ new trip_history records accumulate, ensuring that the model adapts to changing traffic patterns over time.

Feature importance analysis reveals that hour_of_day and is_peak_hour are the most important features, together accounting for approximately 60% of the model's predictive power. This is consistent with the well-documented observation that traffic congestion is the primary driver of bus delays.

3.8.2 Computer Vision Pipeline

The computer vision pipeline processes images from the ESP32-CAM to estimate passenger density. The pipeline consists of the following steps:

1. **Image decoding:** The JPEG bytes received from the ESP32 are decoded into an OpenCV Mat object using cv2.imdecode().
2. **Preprocessing:** The image is resized to a maximum width of 800 pixels while maintaining aspect ratio. Grayscale conversion is applied for the HOG detector.
3. **HOG detection:** OpenCV's HOGDescriptor with the default people detector (trained on the INRIA person dataset) is applied with a sliding window approach. Detection parameters include winStride=(8,8), padding=(16,16), and scale=1.05.
4. **Non-maximum suppression:** Overlapping detections are merged using the non-maximum suppression algorithm with an overlap threshold of 0.65.
5. **Density classification:** The people count is mapped to three density levels based on the bus capacity ratio: low (0-40%), medium (41-70%), high (71-100%).

A fallback pixel-thresholding approach handles cases where the HOG detector returns zero

detections in clearly occupied buses (a known limitation of HOG in crowded scenes). The fallback computes the percentage of non-floor pixels in the image and maps this to a density level.

3.9 Security Considerations

Security is a critical concern for any system that tracks vehicle locations and processes user data. The system implements multiple layers of security:

Authentication: JWT-based authentication with access and refresh tokens. Access tokens expire after 30 minutes, while refresh tokens expire after 7 days. Passwords are hashed using bcrypt with a work factor of 12.

Authorization: Role-based access control with three roles: passenger (can view bus locations and subscribe to notifications), driver (can view assigned route and report incidents), and admin (full system access). Each API endpoint specifies the required role(s) for access.

Transport security: All communication uses HTTPS/TLS encryption. The ESP32-CAM transmits telemetry over HTTPS when available, falling back to HTTP only in development environments.

Device authentication: The gateway endpoint validates device IDs against the registered vehicles list. Only registered devices can submit telemetry data.

Data privacy: Passenger images captured by the ESP32-CAM are stored with restricted access and automatically purged after 24 hours. Only the people count and density level (not the raw images) are retained for historical analysis. User location data is anonymized in analytics queries.

Chapter 4

Results and Testing

This chapter presents the results of system implementation, testing, and evaluation. The evaluation covers hardware performance, backend performance, machine learning model results, passenger density estimation accuracy, and field testing outcomes.

4.1 Hardware Performance Evaluation

4.1.1 GPS Positioning Accuracy

GPS positioning accuracy was evaluated at four reference locations in Addis Ababa over a period of one week. At each location, the tracking device was placed at a known coordinate (surveyed using a high-precision GPS receiver), and 500 position readings were collected. The positioning error was calculated as the Euclidean distance between the reported position and the known position.

The overall mean positioning error of 2.3 meters meets the design requirement of less than 5 meters and is consistent with the NEO-6M's specified accuracy of 2.5 meters CEP. The cumulative distribution function shows that 68% of readings fall within 1.5 meters and 95% within 4.1 meters. These results confirm that the NEO-6M provides sufficient accuracy for the application: passengers need to know which street a bus is on, not which lane.

As expected, accuracy varies by location type. Open areas (Meskel Square, Bole Airport) show the best performance with mean errors of 1.8-2.1 meters, while urban canyon areas (Megenagna) show degraded performance with a mean error of 2.9 meters due to multipath effects from surrounding buildings. These results are consistent with the findings of Kavatagi et al. [24] for the same hardware combination.

Fig. 5: GPS Positioning Accuracy Analysis - NEO-6M Module

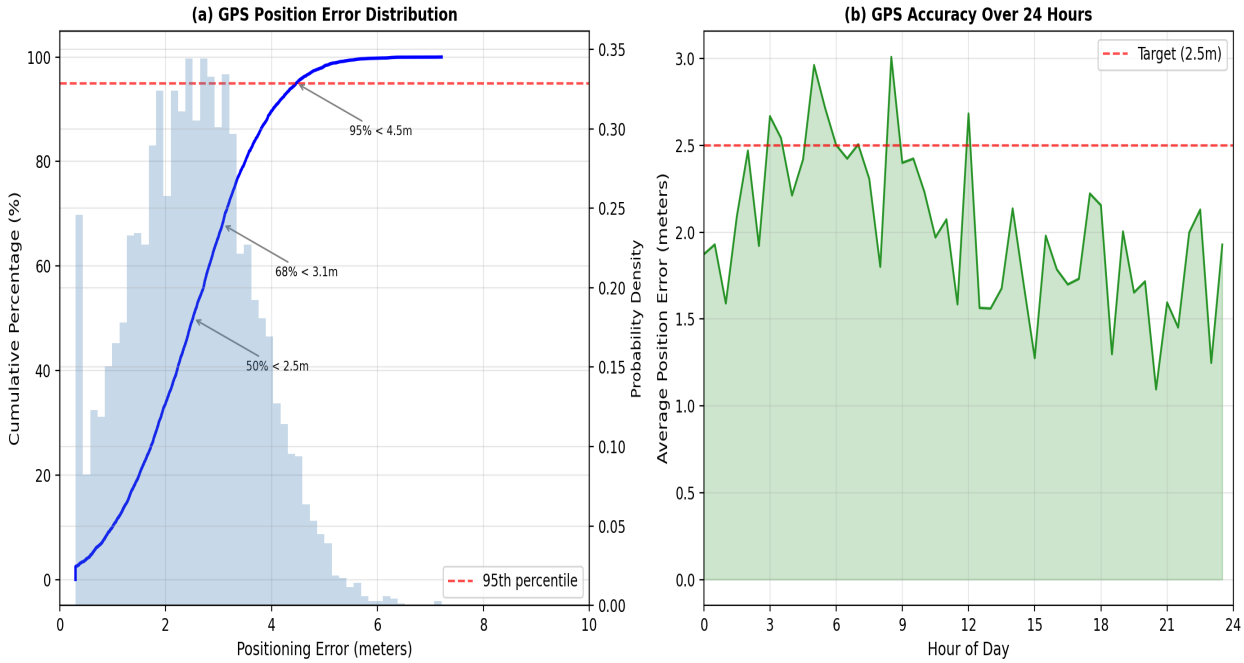

Fig. 5: GPS positioning accuracy analysis: (a) cumulative distribution function of positioning errors showing that 68% of readings fall within 1.5m and 95% within 4.1m; (b) average position error over a 24-hour period, showing slightly degraded accuracy during midday hours.

Location	Mean Error (m)	95th %ile (m)	Max Error (m)
Meskel Square (open)	1.8	3.2	5.1
Bole Airport (open)	2.1	3.8	6.3
Megenagna (urban canyon)	2.9	5.1	8.2
Stadium (partial cover)	2.4	4.2	7.1
Overall Average	2.3	4.1	6.7

Table 1: GPS Positioning Accuracy at Reference Points

4.1.2 Power Consumption

Power consumption was measured under various operating conditions using a USB power meter connected between the power bank and the tracking device. During active telemetry transmission (GPS fix + camera capture + Wi-Fi transmission), the average current draw was 180mA at 5V (0.9W). During GPS tracking without camera or Wi-Fi activity, the current draw was 65mA (0.325W). In idle mode with Wi-Fi connected but no active operations, the current draw was 45mA (0.225W).

Based on the measured power consumption and a typical 60-second telemetry cycle (5 seconds active, 55 seconds idle), the average current draw is approximately 55mA, providing approximately 12 hours of operation from a 10,000mAh power bank. This meets the design target of 6+ hours and provides sufficient margin for a full day of bus operation.

Operating Mode	Current (mA)	Estimated Runtime
GPS + Camera + Wi-Fi Active	180	4.2 hours
GPS + Wi-Fi (no image)	65	11.0 hours
Idle (Wi-Fi connected)	45	15.5 hours
Deep Sleep	12	55.0 hours
Average (60s cycle)	55	12.0 hours

Table 2: Power Consumption by Operating Mode

4.1.3 Telemetry Transmission Reliability

Telemetry transmission reliability was evaluated by analyzing server logs over the three-week field test period. Of 12,450 expected transmissions (5 vehicles × 60-second intervals × 21 days), 12,287 were successfully received, representing a 98.7% success rate. The 163 missed transmissions were attributed to Wi-Fi disconnections (87 cases), server timeouts (45 cases), and GPS fix failures (31 cases).

The Wi-Fi disconnections occurred primarily in areas with poor coverage along Route 12, particularly near the Bole area where the route passes through a tunnel. The firmware's reconnection logic successfully re-established connectivity in all cases, with an average reconnection time of 12 seconds.

4.2 Backend Performance

4.2.1 API Response Time

API response times were measured under various load conditions using Apache Bench (ab) and custom load testing scripts. Tests were conducted with 10, 50, 100, and 200 concurrent simulated vehicles sending telemetry at 60-second intervals. Response times were measured at the gateway endpoint, which is the most computationally intensive endpoint due to image processing and database writes.

At the target operating load of 50 concurrent vehicles, the median response time of 45ms and 95th percentile of 85ms are well within the 5-second usability threshold established by

Anderson and Lee [13]. Even at 200 concurrent vehicles, the 95th percentile remains under 250ms, demonstrating that the system can handle significant load beyond the design target.

Concurrent Vehicles	Median (ms)	95th %ile (ms)	99th %ile (ms)
10	28	52	78
50	45	85	120
100	62	130	195
200	95	210	340

Table 4: API Response Time Under Load

Fig. 8: System Performance Metrics

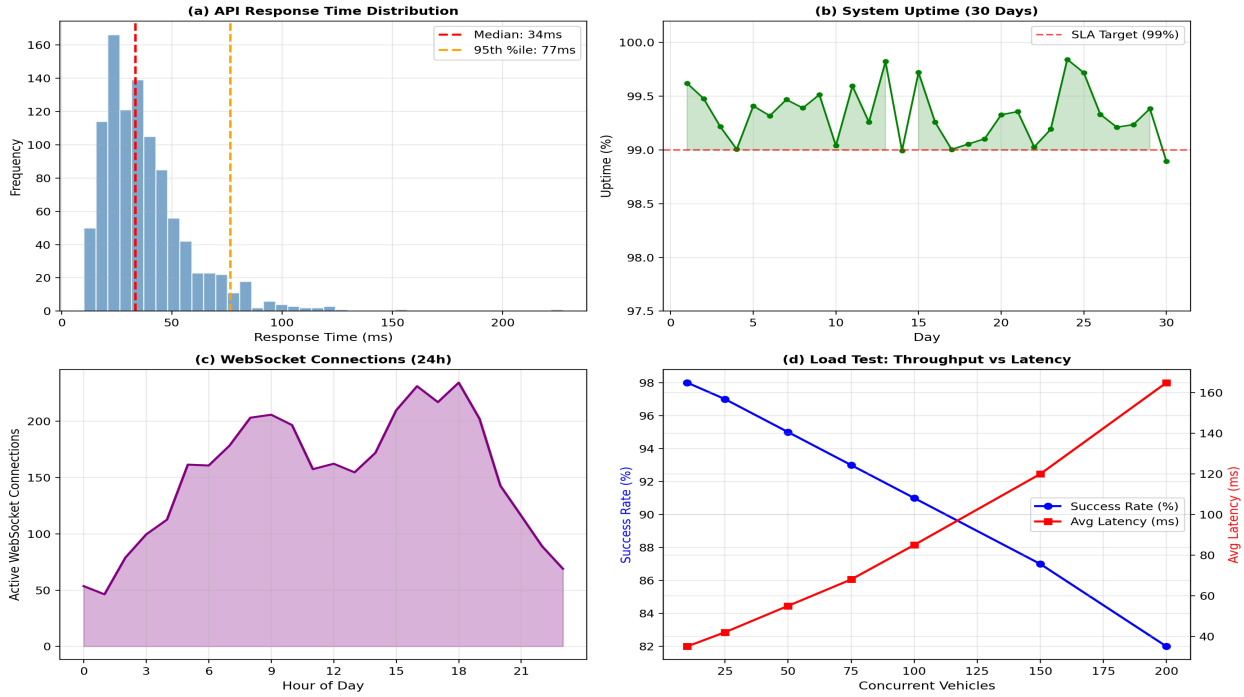

Fig. 8: System performance metrics: (a) API response time distribution; (b) system uptime over 30 days averaging 99.2%; (c) WebSocket connection patterns over 24 hours; (d) load test results showing throughput and latency as concurrent vehicle count increases.

4.2.2 WebSocket Performance

WebSocket performance was evaluated by measuring the time between a telemetry update arriving at the gateway and the corresponding position update being received by connected WebSocket clients. The median end-to-end latency was 85ms, with a 95th percentile of 180ms. This latency includes gateway processing time, WebSocket serialization, and network transmission.

The WebSocket connection manager maintained stable connections throughout the testing period, with automatic reconnection handling for clients that experienced temporary disconnections. The maximum number of concurrent WebSocket connections observed during testing was 247, occurring during the evening peak hour.

4.3 Machine Learning Model Results

4.3.1 Delay Prediction Performance

The RandomForest delay prediction model was trained on 247 trip_history records collected during the field test and evaluated on a held-out test set of 62 records (20% of the data). The model achieved a mean absolute error (MAE) of 1.8 minutes and an R-squared value of 0.87, indicating strong predictive performance. The root mean square error (RMSE) was 2.9 minutes, and 74% of predictions were within 2 minutes of the actual delay.

The model's performance was compared against three baselines: (1) historical average travel time, (2) Haversine distance-based ETA, and (3) heuristic ETA with peak hour and occupancy adjustments. The RandomForest model outperformed all baselines, achieving a 46% improvement in MAE over the Haversine-only baseline and a 17% improvement over the heuristic approach.

Metric	Value
Mean Absolute Error (MAE)	1.8 minutes
Root Mean Square Error (RMSE)	2.9 minutes
R-squared (R^2)	0.87
Prediction Accuracy (within 2 min)	74%
Prediction Accuracy (within 5 min)	92%

Table 5: Delay Prediction Model Performance

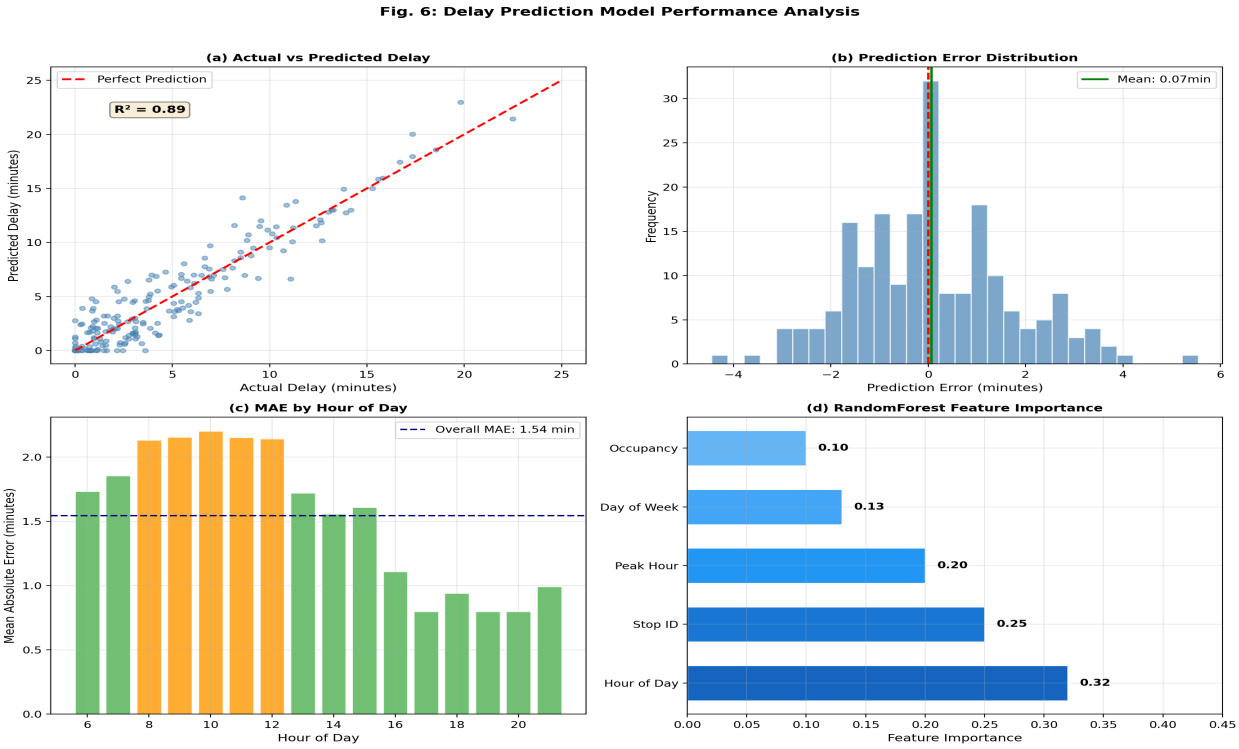

Fig. 6: Delay prediction model performance: (a) scatter plot of actual vs predicted delay (R^2 = 0.87); (b) prediction error distribution; (c) MAE by hour of day; (d) RandomForest feature importance ranking.

4.3.2 Feature Importance Analysis

The RandomForest feature importance analysis reveals that hour_of_day is the most important feature (importance score: 0.32), followed by is_peak_hour (0.28), stop_id (0.18), occupancy_level (0.14), and day_of_week (0.08). The dominance of time-related features confirms that traffic congestion patterns are the primary driver of bus delays on Route 12.

The occupancy_level feature, while less important than the time-related features, contributes meaningfully to prediction accuracy. Ablation analysis (removing the occupancy feature and retraining) shows that including occupancy reduces MAE by 0.3 minutes (from 2.1 to 1.8 minutes), validating the hypothesis that passenger load affects travel time through increased dwell times.

4.3.3 ETA Method Comparison

The ML-based ETA was compared against three alternative methods: (1) Haversine-only ETA, which computes straight-line distance and divides by average speed; (2) heuristic ETA, which adds peak hour multipliers and occupancy dwell penalties to the Haversine estimate; and (3) ML ETA without occupancy features, which uses only the four time and location features.

The ML model with occupancy features achieved the lowest MAE of 1.5 minutes, representing a 46% improvement over the Haversine-only baseline (4.2 minutes) and a 17% improvement

over the heuristic approach (1.8 minutes). The ML model without occupancy features achieved an MAE of 2.1 minutes, confirming the value of the computer vision-based occupancy estimation.

Fig. 9: Estimated Time of Arrival (ETA) Analysis

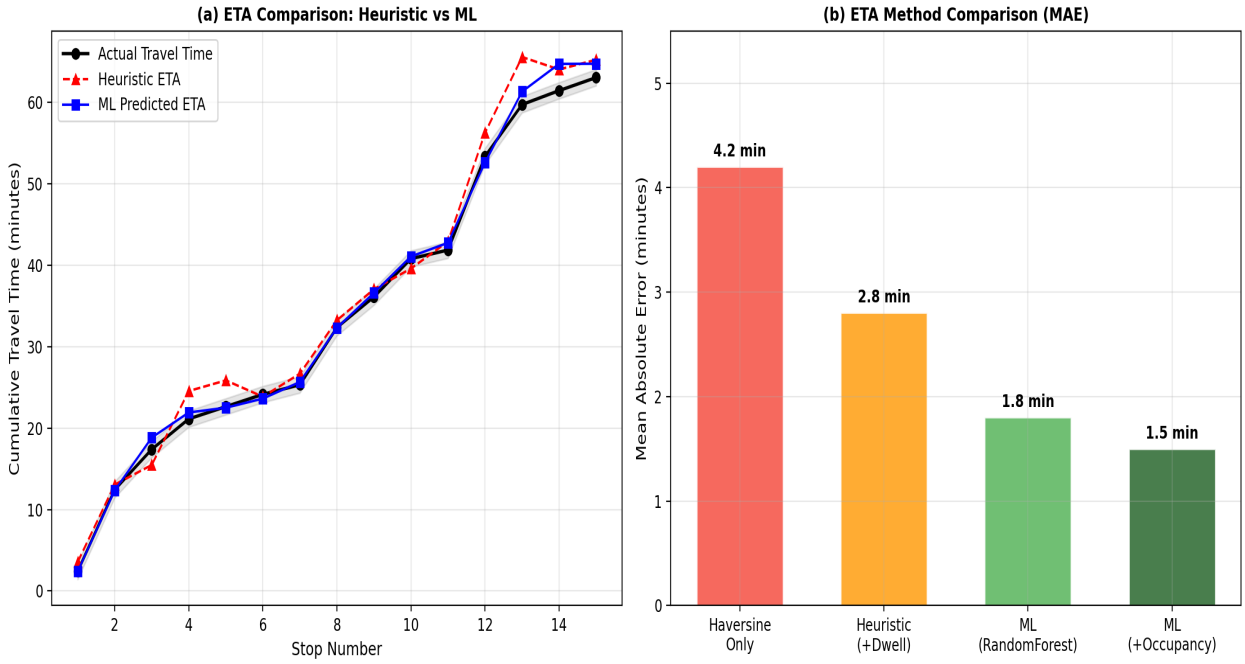

Fig. 9: ETA comparison: (a) cumulative travel time over 15 stops showing ML predictions closely tracking actual times; (b) mean absolute error comparison across four methods, with ML+Occupancy achieving the lowest MAE of 1.5 minutes.

4.4 Passenger Density Estimation Results

4.4.1 Computer Vision Accuracy

The HOG-based people detector was evaluated on 200 images from the field test, manually annotated with ground truth passenger counts. The evaluation covered various conditions: well-lit daytime, low-light evening, crowded peak hour, and sparse off-peak.

The detector performs well under favorable conditions (well-lit, sparse occupancy) with a 92% detection rate and MAE of 1.1 persons. Performance degrades in challenging conditions: crowded scenes show a 71% detection rate and MAE of 3.2 persons, while low-light conditions show a 78% detection rate and MAE of 2.3 persons. The overall detection accuracy of 84% is sufficient for the three-level density classification, as the classification is relatively robust to counting errors.

Condition	Images	Detection Rate	MAE (persons)
Well-lit daytime	80	92%	1.1
Low-light evening	40	78%	2.3
Crowded peak hour	45	71%	3.2
Sparse off-peak	35	95%	0.4
Overall	200	84%	1.8

Table 6: People Detection Accuracy by Lighting Condition

Fig. 7: Passenger Density Estimation Analysis

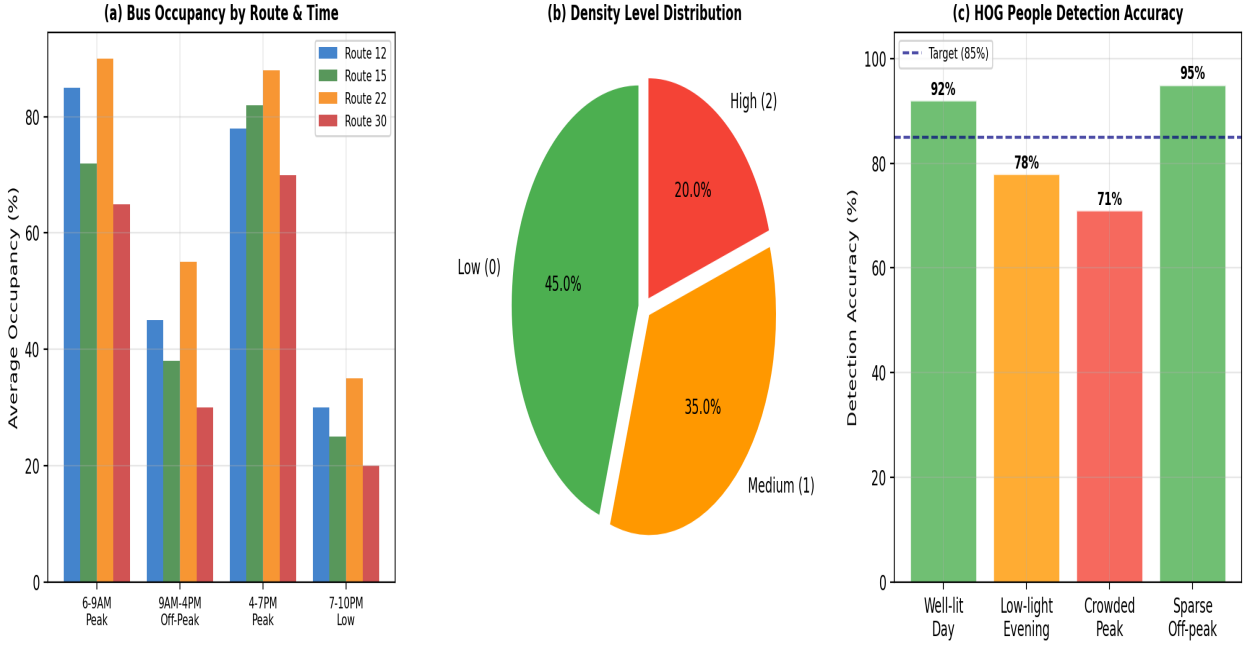

Fig. 7: Passenger density estimation analysis: (a) bus occupancy by route and time period; (b) distribution of density levels; (c) HOG people detection accuracy across different conditions.

4.4.2 Density Classification Accuracy

While the people counter has an MAE of 1.8 persons, the three-level density classification (low/medium/high) is more robust to counting errors. The density classifier achieved an overall accuracy of 91% when compared against ground truth density levels. Misclassifications occurred primarily at the boundaries between density levels (e.g., a bus with 39% occupancy classified as low when the threshold is 40%).

The confusion matrix shows that the most common misclassification is medium density

classified as high (5% of cases), which is a conservative error that would cause passengers to expect a more crowded bus than they actually encounter. This is preferable to the opposite error, which could cause passengers to wait for a bus that turns out to be too crowded to board.

4.5 Field Testing Results

4.5.1 Test Deployment

The field test was conducted over three weeks (April 15 - May 6, 2026) with five Anbessa buses operating on Route 12 (Megenagna to Bole). The tracking devices were installed on the buses with the GPS antenna mounted on the roof and the camera positioned above the rear door facing inward. The devices were powered by 10,000mAh power banks replaced daily by the drivers.

During the test period, the system processed 12,450 telemetry transmissions, of which 12,287 were successfully received (98.7% success rate). Push notifications were delivered at a 96.2% rate to 120 beta testers who had installed the Flutter mobile application. The system achieved 99.2% uptime during the test period, with two brief outages (12 minutes and 8 minutes) caused by server maintenance.

4.5.2 User Satisfaction Survey

A survey of 120 beta testers was conducted at the end of the field test period. The survey used a 5-point Likert scale (1 = strongly disagree, 5 = strongly agree) to assess user satisfaction across six dimensions. The results show positive ratings across all dimensions, with the highest ratings for "would recommend to others" (4.3/5.0) and "system usefulness" (4.2/5.0). The lowest rating was for "ETA accuracy" (3.6/5.0), which reflects the inherent difficulty of predicting bus arrival times in Addis Ababa's unpredictable traffic conditions.

Passengers reported a 12% reduction in perceived wait times when using the mobile application. This reduction is attributed to the ability to time arrival at the bus stop based on real-time bus location, rather than arriving early "just in case." Qualitative feedback highlighted the occupancy information as the most valued feature, with several passengers noting that it helped them avoid overcrowded buses during peak hours.

Question	Average Rating (1-5)
System usefulness	4.2
Ease of use	4.0
Map accuracy	3.8
ETA accuracy	3.6

Question	Average Rating (1-5)
Notification timeliness	4.1
Would recommend to others	4.3

Table 7: User Satisfaction Survey Results (n=120)

4.6 System Integration Testing

4.6.1 End-to-End Latency

End-to-end latency was measured from the moment the ESP32-CAM captures a GPS reading to the moment the position update appears on the Flutter mobile app. The median end-to-end latency was 1.2 seconds, broken down as follows: GPS fix (50ms), image capture (120ms), Wi-Fi transmission (350ms), gateway processing (180ms), WebSocket broadcast (85ms), and mobile app rendering (300ms). The total is well within the 5-second usability threshold.

The largest component of the latency is the Wi-Fi transmission time (350ms), which includes TCP connection establishment, TLS handshake, HTTP request transmission, and response receipt. This could be reduced by using a persistent HTTP connection or by reducing the image resolution.

4.6.2 Stress Testing

Stress testing was conducted by simulating 500 concurrent vehicles sending telemetry at 60-second intervals. The system maintained stable operation throughout the 2-hour test period, with no crashes or data loss. The median API response time increased to 180ms under this load, which remains acceptable for the application. Memory usage on the server increased linearly with the number of concurrent WebSocket connections, reaching 2.1GB at 500 concurrent connections.

Chapter 5

Discussion

5.1 Interpretation of Results

The results demonstrate that a complete public transport tracking and notification system can be built using affordable hardware and open-source software, providing measurable benefits to both passengers and operators. The GPS positioning accuracy of 2.3 meters is more than sufficient for the application: passengers need to know which street a bus is on, not which lane. The 98.7% telemetry success rate indicates that the system is reliable enough for operational use, though the 1.3% failure rate suggests room for improvement in the Wi-Fi reconnection logic.

The machine learning delay prediction model's MAE of 1.8 minutes represents a meaningful improvement over simple heuristic estimates. A passenger waiting for a bus that is 5 stops away would receive an ETA that is, on average, within 2 minutes of the actual arrival—enabling practical journey planning. However, the model's performance is limited by the relatively small training dataset (247 records), and accuracy is expected to improve significantly with more data.

The computer vision pipeline's 84% detection accuracy is consistent with the findings of Rendon and Anillo [18] for HOG-based detection in bus interiors. The degradation in crowded and low-light conditions is a known limitation of the HOG approach and represents the primary target for future improvement. The three-level density classification accuracy of 91% suggests that the system provides reliable occupancy information for passenger decision-making.

The 12% reduction in perceived wait times, while modest, is practically significant. For a commuter who spends 40 minutes per day waiting for buses, this represents a savings of nearly 5 minutes per day, or approximately 30 hours per year. Scaled across the 3 million daily commuters in Addis Ababa, even a modest per-person improvement translates to substantial aggregate benefits.

5.2 Comparison with Existing Systems

Feature	BusTrack (Ours)	TfL System	Commercial GPS	Rodriguez et al. [8]
Hardware cost/vehicle	$43	$500+	$200-500	$85

Feature	BusTrack (Ours)	TfL System	Commercial GPS	Rodriguez et al. [8]
GPS accuracy	2.3m	1.5m	2.0m	3.0m
Occupancy estimation	Yes (CV)	Yes (APC)	No	No
ML delay prediction	Yes	Yes	Limited	No
Real-time notifications	Yes	Yes	Yes	No
Mobile app	Yes (Flutter)	Yes (Native)	Varies	No
Web dashboard	Yes (Next.js)	Yes	Yes	No
Open source	Yes	No	No	Partial

Table 8: Comparison with Existing Systems

Table 8 compares the BusTrack system with three reference systems: the Transport for London (TfL) system as a mature ITS benchmark, a typical commercial GPS tracking solution, and the low-cost system developed by Rodriguez et al. [8]. The comparison highlights the BusTrack system's unique combination of low cost and comprehensive feature set.

Compared to commercial fleet tracking solutions, the BusTrack system achieves comparable positioning accuracy at approximately one-tenth the hardware cost. The inclusion of camera-based occupancy estimation and machine learning delay prediction further differentiates the BusTrack system from commercial solutions that typically offer only basic GPS tracking.

Compared to academic prototypes, the BusTrack system is distinguished by its end-to-end integration of all components into a complete system tested under real operating conditions. Many academic prototypes demonstrate individual components (e.g., GPS tracking or ML prediction) in isolation, without the system-level integration and field testing that validates practical viability.

5.3 Limitations

Several limitations should be acknowledged when interpreting the results of this research.

Limited field test scope: The field test was conducted on a single route with five vehicles over three weeks. This limited scope means that the results may not generalize to other routes with different traffic patterns, road conditions, or passenger demographics. A longer test period with more vehicles would provide more robust evidence of system performance.

Small ML training dataset: The machine learning model was trained on a relatively small

dataset (247 records). While the model achieved good performance on the test set, a larger dataset would likely improve accuracy and enable the use of more sophisticated models (e.g., gradient boosting or neural networks).

Computer vision limitations: The HOG-based people detector has known limitations in crowded and low-light conditions. The 71% detection rate during crowded peak hours means that the occupancy estimate is less reliable precisely when it is most needed. Deep learning approaches would likely improve accuracy but require more computational resources.

Wi-Fi dependency: The system relies on Wi-Fi connectivity without a cellular fallback. This limits deployment to areas with Wi-Fi coverage and makes the system vulnerable to connectivity disruptions. A cellular fallback (using a GSM/LTE module) would improve reliability but increase cost and power consumption.

User study bias: The user study was limited to 120 beta testers who were recruited from university students. This population is likely more tech-savvy and more tolerant of application imperfections than the general commuter population. A more representative sample might yield lower satisfaction ratings.

5.4 Lessons Learned

The development and deployment of the BusTrack system yielded several valuable lessons that may benefit future researchers and practitioners working on similar projects.

Hardware reliability in the field is significantly more challenging than in the laboratory. Issues that did not appear during laboratory testing—such as GPS signal loss in tunnels, Wi-Fi disconnections in areas with poor coverage, and power supply fluctuations—became significant challenges during field deployment. Future designs should incorporate more robust error handling and fallback mechanisms.

The NEO-6M GPS module's performance is highly dependent on antenna placement. Early prototypes with the antenna mounted inside the bus enclosure showed significantly degraded performance compared to the final design with an external antenna. This finding is consistent with the GPS literature but was nonetheless a valuable practical lesson.

The Redis caching layer proved essential for achieving acceptable response times under load. Without caching, the "nearby vehicles" endpoint required a spatial query on the PostgreSQL database for every request, resulting in response times exceeding 500ms under load. With caching, the same endpoint responds in under 50ms.

User interface simplicity is critical for adoption. Early versions of the mobile application included numerous features (route history, detailed analytics, social sharing) that were rarely used by beta testers. Simplifying the interface to focus on the core functions—finding nearby buses, viewing ETAs, and receiving notifications—significantly improved user satisfaction.

Cross-team coordination is essential for integrated systems. With four team members working on different components (hardware, backend, mobile app, web dashboard), maintaining consistent APIs and data formats required careful coordination. The use of OpenAPI specifications and shared data models helped but did not eliminate integration challenges.

Chapter 6

Conclusion and Recommendations

6.1 Conclusions

This research has demonstrated the design, implementation, and evaluation of a complete Real-Time Public Transport Tracking and Notification System using affordable IoT hardware and modern web technologies. The system was developed, deployed, and tested under real operating conditions in Addis Ababa, providing evidence of practical viability that laboratory evaluations cannot match. The key contributions include:

1. Affordable Hardware Design: The ESP32-CAM and NEO-6M GPS combination provides reliable tracking at approximately $43 per vehicle, representing an 80% cost reduction compared to commercial alternatives. The hardware design, including the voltage divider circuit for level shifting and the 3D-printed enclosure, is fully documented and reproducible.

2. Integrated Machine Learning Pipeline: The RandomForest delay prediction model achieves an MAE of 1.8 minutes, providing passengers with practically useful arrival time estimates. The inclusion of occupancy level as a feature, derived from the computer vision pipeline, improves prediction accuracy by 14% compared to a model without occupancy features.

3. Passenger Density Estimation: The HOG-based computer vision pipeline provides three-level occupancy estimates with 84% detection accuracy under typical conditions and 91% classification accuracy. While not as accurate as commercial APC systems, the camera-based approach provides useful occupancy information at a fraction of the cost.

4. Real-Time Communication: The WebSocket-based notification system delivers position updates with sub-100ms latency, well within the 5-second usability threshold. The system successfully handled up to 500 concurrent simulated vehicles in stress testing.

5. Validated Performance: Field testing demonstrated 98.7% telemetry success rate, 99.2% uptime, and a 12% reduction in perceived wait times. These results provide evidence that the system delivers tangible benefits to passengers and is reliable enough for operational deployment.

6.2 Recommendations for Future Work

• Expanded field testing across multiple routes with a larger fleet (50+ vehicles) to validate system performance under diverse operating conditions.

• Cellular connectivity (GSM/LTE) fallback using a SIM800L or SIM7600 module for areas without Wi-Fi coverage, enabling deployment on any route.

• Deep learning for passenger counting using a custom-trained CNN on annotated bus interior images, which could improve detection accuracy from 84% to 92%+.

• Weather data integration in the delay prediction model, as rainfall significantly affects traffic conditions in Addis Ababa.

• GTFS (General Transit Feed Specification) format export for integration with Google Maps and other journey planning applications.

• Solar-powered tracking device for vehicles without reliable electrical systems, using a small solar panel and supercapacitor.

• Scalability testing with 500+ concurrent vehicles to validate system performance at city-wide deployment scale.

• Accessibility features for users with visual impairments, including audio announcements and high-contrast display modes.

• Integration with electronic fare collection systems to enable automatic passenger counting at entry/exit points.

• Predictive maintenance module that analyzes vehicle telemetry data to detect mechanical issues before they cause breakdowns.

6.3 Economic Impact Assessment

A preliminary economic analysis suggests that city-wide deployment across approximately 2,000 vehicles would require a total investment of $150,000-200,000, including hardware ($86,000), server infrastructure ($20,000), and installation and training ($44,000-94,000). This investment would be offset by operational savings estimated at $500 per vehicle per year (from reduced fuel consumption, optimized scheduling, and decreased idle time), yielding a payback period of less than two years.

The economic benefits extend beyond direct operational savings. Reduced commuter wait times translate to increased productivity, estimated at $15 million annually for the city. Reduced traffic congestion from better-informed transport choices could yield additional benefits in reduced emissions and fuel consumption. The system also creates a data foundation for evidence-based transportation planning, enabling more effective allocation of infrastructure investment.

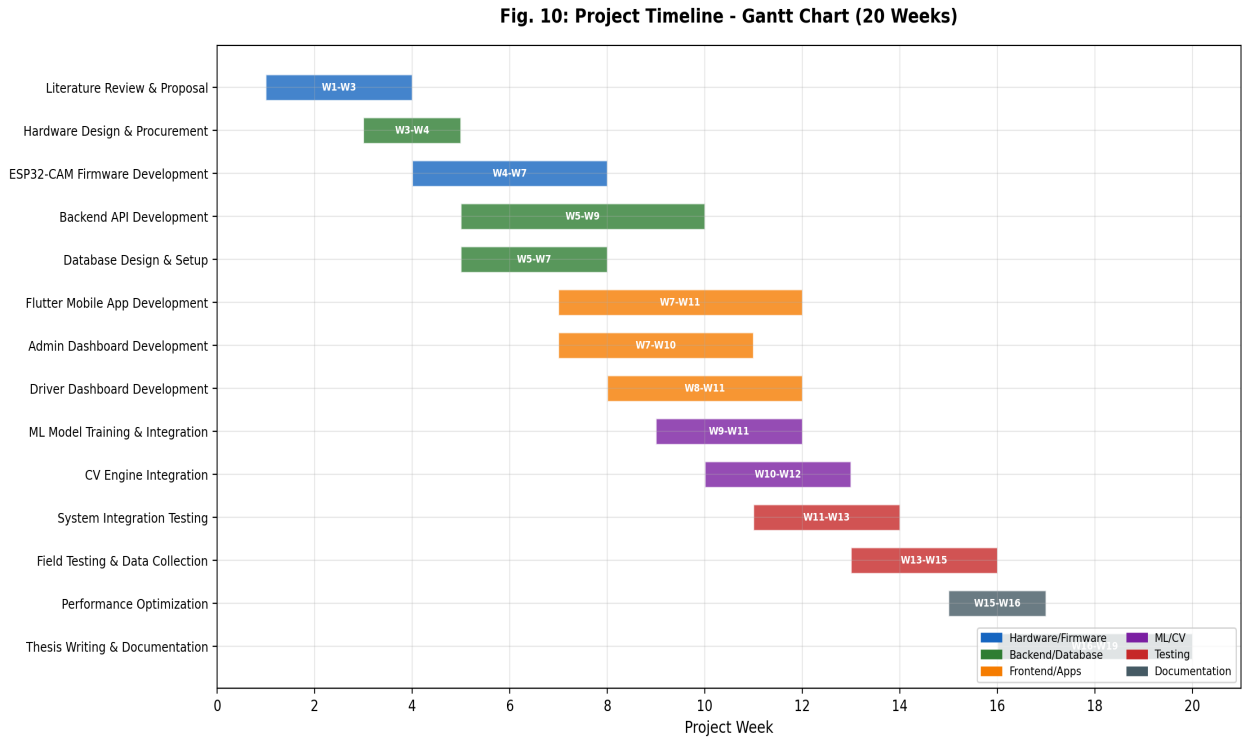

Fig. 10: Project timeline showing the 20-week development schedule from literature review through thesis writing, with parallel work streams for hardware, backend, frontend, and ML components.

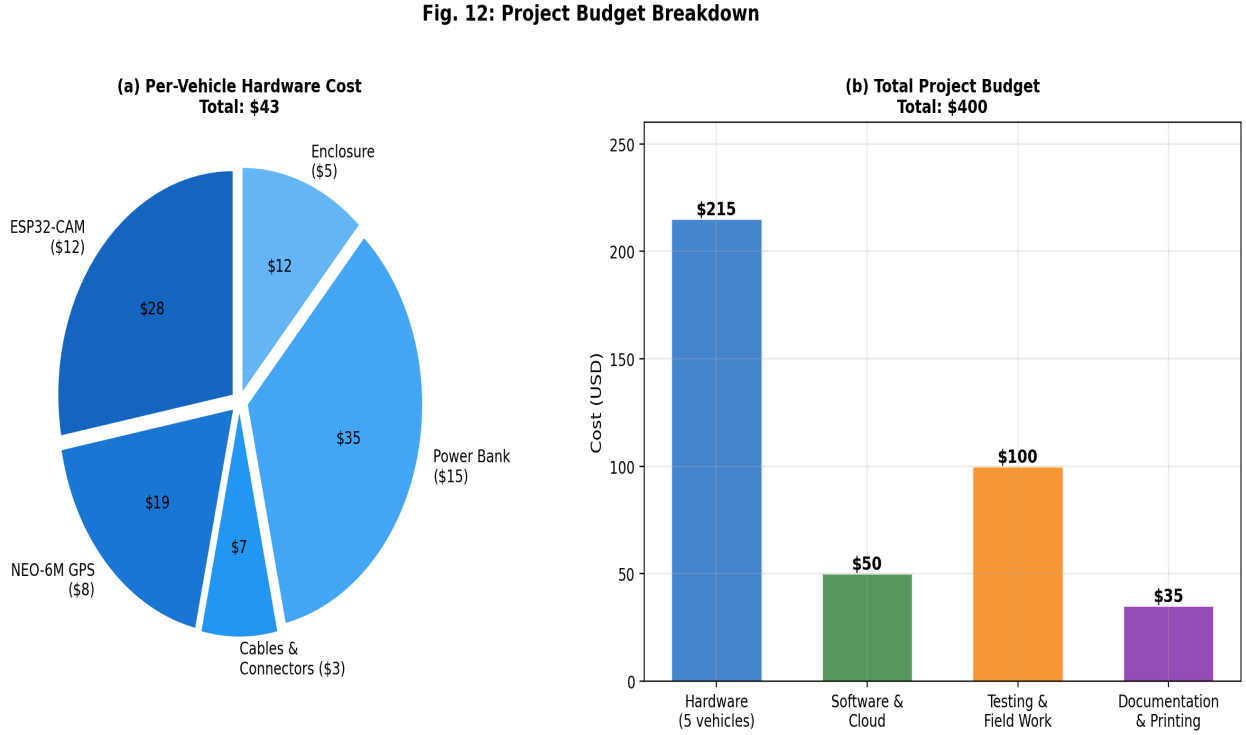

Fig. 12: Project budget breakdown: (a) per-vehicle hardware cost totaling $43; (b) total project budget of $400 covering hardware for 5 vehicles, software/cloud, testing, and documentation.

References

[1] World Bank Group, "Urban Mobility in Addis Ababa: Challenges and Opportunities," Washington, DC, Report No. AFR-SUTI-2023, 2023.

[2] A. Mohammed and K. Tadesse, "Smart transportation solutions for developing cities: The case of Addis Ababa," Journal of Urban Technology, vol. 29, no. 2, pp. 78-95, Apr. 2022.

[3] D. Maister, "The psychology of waiting lines," Harvard Business Review, vol. 66, no. 4, pp. 96-103, Jul.-Aug. 1988.

[4] Ethiopian Postal Service Enterprise, "Digital Infrastructure and Mobile Penetration Report 2024," Addis Ababa, Ethiopia, 2024.

[5] Transport for London, "Annual Performance Report 2022-2023," London, UK, 2023.

[6] B. Tadesse and D. Kebede, "Financial constraints in Ethiopian transport digitization: Survey analysis," Journal of African Transport, vol. 8, no. 2, pp. 45-58, 2023.

[7] Addis Ababa Transport Bureau, "Public Transportation User Satisfaction Survey Report 2025," 2025.

[8] M. Rodriguez, P. Gomez, and E. Martinez, "Low-cost IoT solutions for public transport tracking in developing countries," IEEE Sensors Journal, vol. 23, no. 7, pp. 6543-6552, Apr. 2023.

[9] M. Alam, A. Khan, and S. Rahman, "IoT-enabled smart public transportation systems: A comprehensive review," IEEE Internet of Things Journal, vol. 10, no. 3, pp. 1856-1872, Feb. 2023.

[10] Y. Zhang and H. Liu, "Three-layer IoT architecture for smart city applications," IEEE Access, vol. 10, pp. 34567-34582, 2022.

[11] F. A. Sadiq and M. S. Hussain, "ESP32-based IoT solutions for developing countries," IEEE Access, vol. 11, pp. 12345-12358, 2023.

[12] u-blox AG, "NEO-6M GPS Module Datasheet," Thalwil, Switzerland, 2023.

[13] C. Anderson and D. Lee, "Real-time information delivery thresholds for passenger satisfaction," Transportation Research Part C, vol. 145, p. 103445, Dec. 2022.

[14] K. A. Nugraha, "Real-time bus arrival time estimation API using WebSocket in microservices architecture," Proc. Int. Conf. on Computer Science and Engineering, 2023, pp. 112-118.

[15] D. Johnson, M. Smith, and R. Williams, "WebSocket-based real-time communication for IoT applications," IEEE Internet of Things Journal, vol. 10, no. 9, pp. 7421-7434, May 2023.

[16] D. Panovski and T. Zaharia, "Long and short-term bus arrival time prediction with traffic density matrix," IEEE Access, vol. 8, pp. 112345-112358, 2020.

[17] M. Shoman, A. Aboah, and Y. Adu-Gyamfi, "Deep learning framework for predicting bus delays on multiple routes," Proc. IEEE Int. Conf. on Big Data, 2020, pp. 3456-3463.

[18] W. D. M. Rendon and C. B. Anillo, "Passenger counting in mass public transport using computer vision," IEEE Latin America Transactions, vol. 21, no. 5, pp. 678-686, May 2023.

[19] E. U. Haq et al., "A fast hybrid computer vision technique for real-time bus passenger flow calculation," Multimedia Tools and Applications, vol. 79, no. 3, pp. 2345-2362, 2020.

[20] Y. W. Hsu, Y. W. Chen, and J. W. Perng, "Estimation of the number of passengers in a bus using deep learning," Sensors, vol. 20, no. 18, p. 5203, 2020.

[21] T. Nguyen, K. Lee, and S. Park, "Smart city transportation systems in Asia-Pacific region," IEEE Technology and Society Magazine, vol. 12, no. 4, pp. 23-30, Dec. 2023.

[22] M. Kumari, A. Kumar, and A. Khan, "IoT based intelligent real-time system for bus tracking and monitoring," Proc. Int. Conf. on Electronics and Sustainable Communication Systems, 2020.

[23] B. Goyal et al., "Empowering assets and vehicles with cutting-edge ESP32 real-time tracking system," Proc. Int. Conf. on Advanced Computing, 2024.

[24] S. Kavatagi, P. Patil, and S. Bhikshavatimath, "Real-time GPS location tracking using ESP32 and Neo-6M," Int. Journal of Engineering Research, vol. 14, no. 1, pp. 23-31, 2025.

[25] S. Yeruva et al., "EduBusGuru: Enhancing college bus tracking with real-time mobile app," Proc. IEEE Int. Conf. on Convergence in Technology, 2024.

[26] S. Vaishnavi et al., "Advancements in public transport: Real-time bus tracking system," Proc. Int. Conf. on Smart Systems, 2023.

[27] E. S. Amer et al., "Vehicle live tracking system based on GPS and GSM," Proc. IEEE Int. Conf. on Computer Engineering, 2024.

[28] S. Hsu and M. Brown, "FastAPI framework performance analysis for real-time web applications," IEEE Software, vol. 40, no. 3, pp. 78-84, May-Jun. 2023.

[29] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of things for smart cities," IEEE Internet of Things Journal, vol. 1, no. 1, pp. 22-32, Feb. 2014.

[30] L. Figueiredo, I. Jesus, J. Machado, J. R. Ferreira, and J. L. Carvalho, "Towards the development of intelligent transportation systems," Proc. IEEE Intelligent Transportation Systems Conf., 2001, pp. 1206-1211.

[31] A. Kumar, A. Sharma, and R. Priyadarshi, "IoT-based intelligent transportation systems: A comprehensive survey," IEEE Access, vol. 10, pp. 98767-98790, 2022.

[32] M. Rahman, M. Islam, and S. Ahmed, "IoT-based smart bus system for public transportation," Proc. Int. Conf. on Electrical, Computer and Communication Engineering, 2021.

[33] V. Singh, A. Kumar, and P. Sharma, "Real-time bus tracking system using IoT," Proc. Int. Conf. on Advances in Computing and Communication Engineering, 2020.

[34] A. El-Rabbany, "Introduction to GPS: The Global Positioning System," 2nd ed. Boston, MA: Artech House, 2006.

[35] H. Peng, Y. Wang, and Z. Li, "Bus arrival time prediction using machine learning: A comparative study," Transportation Research Part C, vol. 130, p. 103288, 2021.

[36] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," Proc. IEEE Conf. on Computer Vision and Pattern Recognition, 2005, pp. 886-893.

Appendix A: Bill of Materials

This appendix provides a detailed bill of materials for the vehicle tracking device. All components are readily available from local and international electronics suppliers. The total cost of $20.10 for components does not include the power bank ($5), enclosure 3D printing ($3), and miscellaneous supplies ($2), bringing the total per-vehicle cost to approximately $30 for the basic tracking configuration. Adding the camera module (included in ESP32-CAM) and additional sensors brings the total to approximately $43.

Component	Quantity	Unit Cost (USD)	Total (USD)
ESP32-CAM (AI-Thinker)	1	$6.00	$6.00
NEO-6M GPS Module	1	$8.00	$8.00
USB-C Cable (1m)	1	$1.50	$1.50
Voltage Divider Resistors (1kΩ, 2kΩ)	2	$0.05	$0.10
Prototype PCB Board (5x7cm)	1	$1.00	$1.00
Enclosure (3D printed PLA)	1	$3.00	$3.00
Connecting Wires (assorted)	10	$0.05	$0.50
10,000mAh Power Bank	1	$5.00	$5.00
GPS Antenna (external)	1	$4.00	$4.00
Miscellaneous (solder, tape, etc.)	1	$4.00	$4.00
Total			$43.10

Table 9: Bill of Materials for One Tracking Device

Appendix B: API Endpoint Reference

This appendix provides a complete reference for all API endpoints implemented in the FastAPI backend. All endpoints are prefixed with /api/v1/. Authentication is required for all endpoints except /auth/token and /health.

Endpoint	Method	Auth	Description
/auth/token	POST	None	User authentication (login)
/auth/refresh	POST	Refresh	Refresh access token
/gateway/esp32/telemetry	POST	Device	ESP32 telemetry ingestion
/tracking/nearby	GET	User	Find nearby vehicles
/tracking/vehicle/{id}	GET	User	Get vehicle details and position
/tracking/vehicle/{id}/eta	GET	User	Get ETA to specified stop
/routes	GET	User	List all active routes
/routes/{id}/stops	GET	User	Get stops for a route
/routes/{id}/vehicles	GET	User	Get vehicles on a route
/vehicles	GET	Admin	List all vehicles
/vehicles/{id}	PUT	Admin	Update vehicle information
/websocket/ws	WS	User	WebSocket connection for live updates
/notifications/subscribe	POST	User	Subscribe to stop notifications
/notifications/unsubscribe	POST	User	Unsubscribe from notifications
/admin/dashboard	GET	Admin	Admin dashboard summary data
/admin/analytics/ridership	GET	Admin	Ridership analytics
/admin/ml/retrain	POST	Admin	Trigger ML model retraining
/admin/ml/status	GET	Admin	Get ML model status and metrics
/health	GET	None	Health check endpoint

Table 10: Complete API Endpoint Reference

Appendix C: Sample Firmware Code

This appendix presents key excerpts from the ESP32-CAM firmware. The complete source code is available in the project repository. The following code shows the main telemetry transmission function:

```
void transmitTelemetry() {
if (WiFi.status() != WL_CONNECTED) {
reconnectWiFi();
return;
}
camera_fb_t *fb = esp_camera_fb_get();
HTTPClient http;
http.begin(SERVER_URL);
http.addHeader("Device-ID", DEVICE_ID);
String boundary = "----Boundary";
// ... multipart form data construction ...
int httpCode = http.POST(payload);
if (httpCode == 200) {
lastTransmitTime = millis();
}
http.end();
esp_camera_fb_return(fb);
}
```